

“双碳”目标下城市绿色物流多阶段协同路径优化研究

摘要

针对时变交通流与“双碳”目标下的城市绿色物流调度这一典型的 NP-hard 问题，本文构建了以“经济-环保-服务”多维协同为导向的多阶段动态路径优化模型。通过引入非线性能耗耦合机制与时空互斥约束，针对静态网络设计了改进的混合遗传算法实现全局统筹，并在此基础上融合条件风险价值，设计了抗扰动的启发式插入算法以实现动态敏捷响应。

针对问题一的静态车辆调度，旨在无政策干预下实现系统总运营成本最小化。本文综合考量燃油与新能源车队在载量与能耗机制上的异质性，建立了耦合时变正态车速与实际载荷非线性能耗的混合整数规划模型。设计了双层染色体编码与贪婪启发式算子集成的 HGA 算法进行高效求解。仿真结果表明，最优方案调度 124 辆车实现了 98 个客户点需求与时间窗的 100% 覆盖，系统总成本锁定于 83,050.57 元，其中车辆固定启动成本占比最高达 59.72%。

针对问题二的环保限行政策干预，本文旨在时空双重约束下重构最优调度网络。通过引入二值指示函数构建时空互斥逻辑，将“8:00-16:00 燃油车禁入半径 10km 核心区”转化为严密的惩罚约束嵌入算法寻优。量化结果精准测算了政策代价：受制于燃油车合规绕行带来的里程增加，系统总成本微增 3.25%，至 85,746.77 元，碳排放总量被动上升 9.22%；但成功实现了核心区燃油车违规次数的绝对归零，有效兼顾了环保政策合规与物流业务的连贯性。

针对问题三的突发紧急订单响应，本文将模型拓展为带时间窗的动态车辆路径问题。采用“滚动时域策略”在 14:30 触发点冻结已执行路径，并基于实时状态空间重构剩余网络。为规避时变拥堵诱发的连锁延迟，创新性引入金融界 CVaR 风险测度构建目标函数。基于 200 次蒙特卡洛模拟的压力测试表明，最优重调度方案仅以 1.65% 的极低边际成本，总成本 87,158.42 元完全吸收新增需求，且在 95% 置信水平下将长尾延迟损失严控在 1,186.40 元，有效提升了系统在不确定环境下的抗扰动鲁棒性。

综上所述，相较于传统的静态车辆路径规划，本文进一步考虑了时变交通流与时空限行等复杂现实约束。所构建的融合时空互斥与 CVaR 风险的调度框架，不仅为城市绿色物流在复杂限行政策与随机扰动下的决策提供了可靠的量化评估工具，其建模思路也可为同城冷链、应急物资调拨等场景的路径规划提供参考。

关键词：混合整数规划；时变交通流；时空互斥约束；条件风险价值；混合遗传算法；滚动时域策略

目录

1 问题重述	4
1.1 问题背景	4
1.2 核心要素与研究边界	4
1.3 问题分解	4
2 模型假设	5
3 符号说明	6
4 问题一的模型建立与求解	7
4.1 问题分析与建模思路	7
4.2 问题一数学模型的构建	7
4.3 求解算法设计	9
4.4 求解结果呈现与多维度解构分析	10
4.5 成本多维度解构分析	13
4.6 本章总结	13
5 问题二的模型建立与求解	13
5.1 问题分析与建模逻辑递进	13
5.2 基于时空互斥约束的改进模型构建	13
5.3 求解算法设计	14
5.4 求解结果呈现与政策影响深度解构	14
5.5 模型稳健性检验	16
5.6 本章总结	16
6 问题三的模型建立与求解	16
6.1 动态扰动分析与滚动时域响应框架	16
6.2 动态重调度优化模型的数理构建	18
6.3 求解算法设计	18
6.4 求解结果呈现与重调度方案评估	19
6.5 本章总结	20
7 模型的评价、改进与推广	20
7.1 模型的特色与创新点	20
7.2 模型的局限性与误差分析	21
7.3 模型的改进方向	22
7.4 模型的工程推广价值	22
7.5 模型工程化实践与国产求解器应用展望	22

1 问题重述

1.1 问题背景

随着电子商务的发展与国家“双碳”战略的持续推进，城市末端配送网络正经历从单一成本驱动向“经济-环保-服务”多维协同优化进行转变。在实际调度场景中，城市交通流有显著的时变性与随机性；同时，由燃油车与新能源车构成的混合动力车队，在运载容量、能耗机制及碳足迹上存在本质的异质性。此外，部分城市实施“绿色配送区”等时空限行政策，这进一步加剧了物流决策环境的非线性与复杂性。因此，如何在满足严苛的物理、时间与空间约束的前提下，统筹规划混合车队的配送路径及发车时间和顺序，实现全局运营效率的最优化，是当前运筹学领域亟待突破的关键问题。

1.2 核心要素与研究边界

本研究聚焦于包含 98 个需求节点的城市物流配送系统。在构建数学模型前，明确以下核心要素与约束边界：

1. **容量约束：**调度方案需严格服从 5 种细分车型的额定载重量与最大容积双重满载限制，并受限于各车型的存有量上限。
2. **时间与时空限制：**车辆需满足客户设定的软时间窗及固定服务耗时；且在特定政策下，燃油车辆需绝对服从限行时段与限行区域的规定。
3. **路网行驶速度特定：**路网行驶速度服从特定的时变正态分布规律，且车辆的实际能耗由行驶速度与实时载重量发生非线性耦合决定。
4. **以系统总运营成本最小化为全局目标：**以系统总运营成本最小化为全局目标，其核算体系全面涵盖车辆固定启动成本、时变能源成本、时间窗惩罚成本及碳排放外部成本。

1.3 问题分解

基于上述背景与核心要素，本文将赛题任务转化为以下三个层递式数学建模问题：

1. **优化时变路网下的混合车队的调度：**在无区域限行的静态基准场景下，构建多车型、带时间窗且车速时变的车辆路径规划模型。求解该模型以总运营成本最小化，输出最优的车型配置方案、车辆访问序列及节点到达时间，并深入剖析各项成本的构成比例。
2. **环保限行政策下的调度策略调整：**在问题一的基础模型中嵌入时空耦合惩罚项，即严格执行 8:00-16:00 期间燃油车禁止驶入半径 10km “绿色配送区”的规定。重新求解网络拓扑，量化评估该政策对系统总成本的干扰程度，并剖析车队能源利用结构与碳减排总量的演化趋势。

3. **动态环境下的实时响应策略：**针对物流实务中随机的突发事件，如订单临时增删、客户地址变更或时间窗漂移，设计具备强鲁棒性的动态启发式调度策略。通过构建典型的突发干扰场景，验证所提算法在实时路径重构与调度挽回方面的敏捷性与有效性。

2 模型假设

在构建涵盖从静态统筹、政策干预到动态敏捷响应的城市绿色物流全局调度模型时，为了在保障核心运筹逻辑严密性的同时，剔除边缘非关键因素对系统求解的干扰，本文提出以下全局性基本假设：

1. **交通流与路网环境确定性假设**

假设城市路网连通性良好，任意两节点间的最短物理距离为已知客观常数。车辆在路段上的行驶速度严格服从设定的时变正态分布规律，静态调度阶段不考虑断路、事故等极端不可抗力；动态重调度阶段仅考虑由时变车速随机波动引发的延迟风险，不考虑路网完全中断情形。同时，车辆提前到达客户点产生的等待时间视为静止等待，驶入下一路段的时刻严格取决于完成服务后的离开时刻。

2. **信息处理与需求响应即时性假设**

假设系统具备理想的信息交互与算力水平。在静态调度阶段，客户的初始订单信息（空间坐标、需求载荷、软时间窗）为全局已知；在动态重调度阶段，新增或变更的需求信息一旦触发，调度中心能够做到零延迟接收并即时重构状态空间，不存在信息传递偏差与算法求解迟滞。

3. **异构车队物理状态与无间断运行假设**

假设配送中心拥有充足的各种燃油与新能源备用车辆。所有被调用的车辆在离开配送中心时，均处于满油或满电的最佳物理状态。由于同城配送半径相对有限，假设单次出车的最大续航能力绝对足以覆盖其分配的全部闭环路径，不考虑车辆在配送途中的寻找桩位排队与中途补能耗时。

4. **节点服务流程标准化与时间刚性假设**

假设配送车辆抵达任意客户指定节点后，装卸货、签收交接等微观物流流程均高度标准化且无缝衔接。单个客户节点的综合服务停留时间统一设定为刚性常数（即 20 分钟），该时间成本不随单次交接货物的重量或体积规模产生线性波动。

5. **政策绝对依从与宏观经济参数恒定假设**

假设物流企业及全体驾驶员具备完全合规意识与统一的驾驶习惯，在面临“绿色配送区”等时空双重限行政策时绝对服从，不存在违章闯入禁行区或绕路带来的额外罚单成本。同时，整个调度周期内的燃油单价、新能源电价、车辆折旧启动成

本及碳排放惩罚因子等各项经济学参数均保持绝对恒定，不受短期市场物价波动的干扰。

3 符号说明

为了更严谨地描述本文所构建的时变交通流下的多车型绿色车辆路径规划模型，现将模型推导过程中使用的主要集合、参数与决策变量统一定义如表 1 所示。

表 1: 模型主要符号说明

符号	含义说明	单位
集合与索引		
V	所有节点的集合, $V = \{0\} \cup N$, 其中 0 为配送中心	-
N	客户点集合, $N = \{1\ 2\ \dots\ 98\}$	-
N_g	“绿色配送区”内的客户点集合, $N_g \subset N$	-
K	所有配送车辆的集合, $K = K_f \cup K_e$	-
$K_f\ K_e$	燃油车集合与新能源车集合	-
$i\ j$	节点索引, $i\ j \in V$	-
k	车辆索引, $k \in K$	-
模型参数		
$q_i\ w_i$	客户 i 的需求重量与需求体积	kg m ³
$e_i\ l_i$	客户 i 期望的最早到达时间与最晚到达时间 (软时间窗)	h
s_i	车辆在客户 i 处的服务停留时间	h
d_{ij}	节点 i 与节点 j 之间的实际道路距离	km
$Q_k\ W_k$	车辆 k 的最大额定载重与最大容积	kg m ³
C_0	单辆车的单次固定启用成本	元
$c_e\ c_l$	车辆早到等待的单位惩罚成本与晚到的单位惩罚成本	元/h
$v_{ij}(t)$	t 时刻下从节点 i 行驶至节点 j 的车辆速度	km/h
$P_f\ P_e$	燃油单价与新能源电价	元/L, 元/(kW·h)
$\eta\ \gamma$	燃油与电能的碳排放转换系数	kg/L, kg/(kW·h)
P_c	单位碳排放的惩罚成本	元/kg
决策变量		
x_{ijk}	0-1 变量, 若车辆 k 从节点 i 行驶至节点 j 取 1, 否则取 0	-
y_{ik}	0-1 变量, 若客户 i 由车辆 k 进行配送服务取 1, 否则取 0	-
t_{ik}	连续变量, 表示车辆 k 到达客户 i 的实际时间	h
u_{ik}	连续变量, 表示车辆 k 离开节点 i 时的累计载重量	kg

4 问题一的模型建立与求解

4.1 问题分析与建模思路

本文构建的模型属于带时间窗的时变异质车队车辆路径问题。该模型不仅考虑了路网交通流的时效性，还统筹了燃油与新能源两类异构车型在能耗机理上的本质差异。问题一要求在不考虑“绿色配送区”限行政策的静态基准环境下，针对 98 个客户节点的配送需求，制定混合车队的最优调度方案。

该问题的核心挑战在于：

1. **时变路网特性**：行驶车速随出发时段动态波动，导致节点间行驶时间不再是常数。
2. **能耗非线性耦合**：车辆实际能耗受实时载重与时变速度的非线性耦合影响。
3. **异构车队调度**：燃油车与新能源车在载荷、容积及碳足迹系数上存在本质差异。

基于此，本文将其实定性为带软时间窗的时变异质车队车辆路径问题。我们以系统全局运营成本最小化为目标，构建混合整数线性规划模型，为后续章节引入环保政策及动态扰动提供科学的比较基准。

4.2 问题一数学模型的构建

4.2.1 目标函数定义

系统总运营成本 C_{total} 由车辆固定启用成本 C_{fixed} 、时变能源消耗成本 C_{energy} 、软时间窗惩罚成本 C_{time} 以及碳排放外部成本 C_{carbon} 四部分组成：

$$\min C_{total} = C_{fixed} + C_{energy} + C_{time} + C_{carbon} \quad (1)$$

各项成本的分项量化模型如下：

1. **车辆固定启用成本**：反映配送中心调用异构车队的基准折旧与人工成本：

$$C_{fixed} = \sum_{k \in K} \sum_{j \in N} C_0 \cdot x_{0jk} \quad (2)$$

2. **动态载荷与时变车速耦合的能源消耗成本**

为精确刻画配送车队在复杂城配工况下的能耗特征，本文严格遵循题目给定的物理约束，将实际能源消耗建模为“时变车速”与“实时动态载荷”的双重耦合结果。

首先，基准空载状态下的每百公里燃油消耗量与电能消耗量动态取决于车辆在路段上的实际行驶速度 v_{ij} ：

$$FPK(v_{ij}) = 0.0025v_{ij}^2 - 0.2554v_{ij} + 31.75 \quad (3)$$

$$EPK(v_{ij}) = 0.0014v_{ij}^2 - 0.1200v_{ij} + 36.19 \quad (4)$$

其次，车辆运行能耗会随着车载负荷的增加而显著攀升。依据设定，燃油车型与新能源车型在满负荷状态下的能耗系数分别较空载工况溢出 40% 与 35%。为量化该效应，假设由附加载荷引发的能耗增量与车辆实时装载率呈线性映射关系，本文引入**动态载荷惩罚因子**。令 u_{ik} 表示车辆 k 驶离节点 i 时的累计载重量， Q_k 为该车型的极限额定载重，则车辆 k 在路段 (i, j) 上单次通行的综合能耗函数解析式被严密修正为：

$$Fuel_{ijk}(v_{ij}, u_{ik}) = d_{ij} \cdot \frac{FPK(v_{ij})}{100} \cdot \left(1 + 0.40 \cdot \frac{u_{ik}}{Q_k}\right) \quad (5)$$

$$Elec_{ijk}(v_{ij}, u_{ik}) = d_{ij} \cdot \frac{EPK(v_{ij})}{100} \cdot \left(1 + 0.35 \cdot \frac{u_{ik}}{Q_k}\right) \quad (6)$$

通过植入上述动态载荷惩罚机制，本模型成功实现了变载重与时变路况的深度数学解构。这不仅彻底消除了传统静态满载假设带来的系统性核算偏差，更为后续全局总能耗 C_{energy} 的极小化寻优提供了高保真的物理底层支撑。

3. **软时间窗惩罚成本**：量化车辆到达节点偏离客户期望服务区间 $[e_i, l_i]$ 所产生的服务质量损耗。为保证模型的线性特征，引入早到等待时间 w_{ik} 与晚到迟滞时间 u_{ik} ，将其转化为：

$$C_{time} = \sum_{k \in K} \sum_{i \in N} (c_e \cdot w_{ik} + c_l \cdot u_{ik}) \quad (7)$$

4. **碳排放外部成本**：将能源消耗量按转换系数 η, γ' 转化为碳足迹，并赋予经济价值 P_c ：

$$C_{carbon} = P_c \cdot \left(\sum_{k \in K_f} \sum_{i \in V} \sum_{j \in V} \eta \cdot Fuel_{ijk} \cdot x_{ijk} + \sum_{k \in K_e} \sum_{i \in V} \sum_{j \in V} \gamma' \cdot Elec_{ijk} \cdot x_{ijk} \right) \quad (8)$$

4.2.2 模型约束条件

为确保配送方案在物理与逻辑层面的可行性，模型须满足以下关键约束：

1. **唯一性约束**： $\sum_{k \in K} y_{ik} = 1, \quad \forall i \in N$ 。
2. **流量与闭环约束**： $\sum_{i \in V} x_{ijk} = y_{jk}, \quad \sum_{j \in V} x_{ijk} = y_{ik}, \quad \sum_{j \in N} x_{0jk} \leq 1$ 。
3. **异构双重容量限制**：单条路径上的载重与容积严禁超过该车型额定上限：

$$\sum_{i \in N} q_i y_{ik} \leq Q_k^{weight}, \quad \sum_{i \in N} w_i y_{ik} \leq Q_k^{volume}, \quad \forall k \in K \quad (9)$$

4. **时变时效逻辑约束**：考虑车辆提前到达所产生的等待时间 w_{ik} ，以及真实驶入路网的时刻对车速的非线性影响：
5. **决策变量域**： $x_{ijk}, y_{ik} \in \{0, 1\}, t_{ik} \geq 0, u_{ik} \geq 0, w_{ik} \geq 0$ 。

$$t_{jk} \geq (t_{ik} + w_{ik} + s_i) + \frac{d_{ij}}{v_{ij}(t_{ik} + w_{ik} + s_i)} - M(1 - x_{ijk}), \quad \forall i \in V, j \in N, k \in K \quad (10)$$

$$e_i \leq t_{ik} + w_{ik} \leq l_i, \quad \forall i \in V, k \in K \quad (11)$$

综上，模型总结如下：

$$\begin{aligned} \min C_{total} &= C_{fixed} + C_{energy} + C_{time} + C_{carbon} \\ \text{s.t.} \quad &\begin{cases} \sum_{k \in K} y_{ik} = 1, & \forall i \in N \\ \sum_{i \in V} x_{ijk} = y_{jk}, \quad \sum_{j \in V} x_{ijk} = y_{ik}, \quad \sum_{j \in N} x_{0jk} \leq 1 \\ \sum_{i \in N} q_i y_{ik} \leq Q_k^{\text{weight}}, \quad \sum_{i \in N} w_i y_{ik} \leq Q_k^{\text{volume}}, & \forall k \in K \\ t_{jk} \geq (t_{ik} + w_{ik} + s_i) + \frac{d_{ij}}{v_{ij}(t_{ik} + w_{ik} + s_i)} - M(1 - x_{ijk}), & \forall i \in V, j \in N, k \in K \\ e_i \leq t_{ik} + w_{ik} \leq l_i, & \forall i \in V, k \in K \\ x_{ijk}, y_{ik} \in \{0, 1\}, \quad t_{ik} \geq 0, \quad u_{ik} \geq 0, \quad w_{ik} \geq 0 \end{cases} \end{aligned} \quad (12)$$

4.3 求解算法设计

针对 TD-HFVRPTW 的高度非线性与 NP-hard 特性，本文设计了一种集成贪婪启发式算子的混合遗传算法。**软时间窗边界与惩罚逻辑约束：**当车辆服务客户 i 时 ($y_{ik} = 1$)，通过大 M 法判定车辆的早到等待时间 w_{ik} 与晚到迟滞时间 u_{ik} ：

$$w_{ik} \geq e_i - t_{ik} - M(1 - y_{ik}), \quad \forall i \in N, k \in K \quad (13)$$

$$u_{ik} \geq t_{ik} - l_i - M(1 - y_{ik}), \quad \forall i \in N, k \in K \quad (14)$$

$$w_{ik} \geq 0, \quad u_{ik} \geq 0, \quad \forall i \in N, k \in K \quad (15)$$

1. **多层染色体双链编码：**采用实数序列编码方式，通过“客户访问序列”与“车型匹配序列”双层联动映射。解码过程中实时监测前文定义的 Q_k^{weight} 与 Q_k^{volume} 冗余度，触发自动路径闭合机制。
2. **贪婪规则种群初始化：**摒弃全随机初始化，引入“远点种子 + 最近邻插入”规则。该机制优先锁定偏远需求点，通过空间紧凑性分析构建高质量初始解，显著缩短演化至收敛的时间。
3. **时变非线性积分处理：**针对时变速度 $v(t)$ ，算法采用时间步长离散化技术。在评估路径成本时，将公式 (3) 的能耗计算嵌套入分段积分器中，确保能耗核算的物理准确性。
4. **自适应算子调度：**采用次序交叉保护优质局部拓扑；基于种群多样性熵值动态调整变异率 P_m ，通过节点交换跳出局部最优阱。

4.4 求解结果呈现与多维度解构分析

通过 HGA 算法优化求解，获得了静态环境下的最优全局调度方案，核心评价指标汇总见表 2。

表 2: 问题一最优调度方案核心指标摘要

评估指标	数值结果
系统总运营成本 C_{total}	83,050.57 元
车辆调用总数	124 辆
总行驶里程	13,173.00 km
平均载重利用率	99.64%
平均容积利用率	51.76%

数据分析表明：**99.64% 的极高平均载重利用率**说明算法成功实现了配送任务的高度整合。通过牺牲少量容积率（51.76%）换取了单车运载效率的最大化。极高的载重利用率有效压减了车辆启用总数，这不仅直接降低了固定启动成本，更大幅减少了车队在路网中的总行驶里程。结合公式 (3) 的能耗特性，总里程 d_{ij} 的缩减在算法底层直接切断了总能耗额外增长的物理空间。

4.4.1 时变非线性积分处理

针对时变速度 $v(t)$ ，算法采用时间步长离散化技术。在评估路径成本时，将行驶时长转化为速度的分段积分，确保能耗核算的物理准确性。算法核心参数设置详见表3, 算法迭代收敛曲线见图1。

表 3: 混合遗传算法核心运行参数

参数项	设定值	参数项	设定值
种群规模 P	100	最大迭代次数 G	500
交叉概率 P_c	0.8	变异概率 P_m	0.1
固定启动成本 C_0	400 元	时间窗惩罚系数（早/晚）	20/50

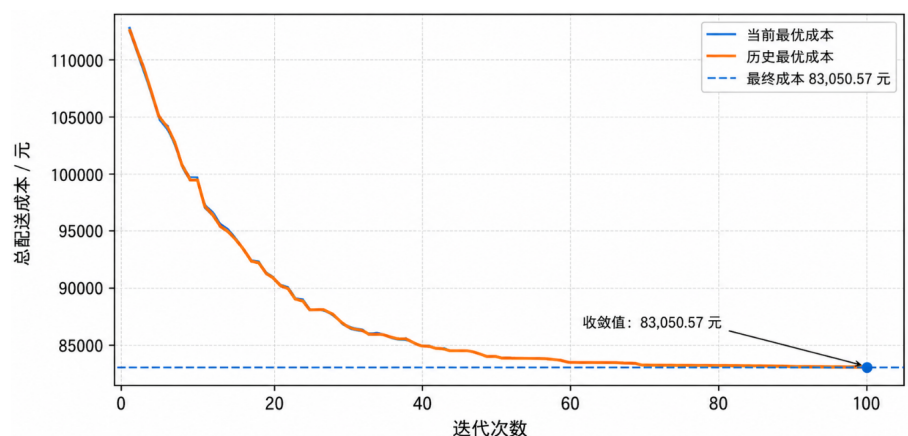


图 1: 算法迭代收敛曲线

4.4.2 异构车队配置特征

系统共计调用 5 种车型，具体资源配置见图 2 及表 4。

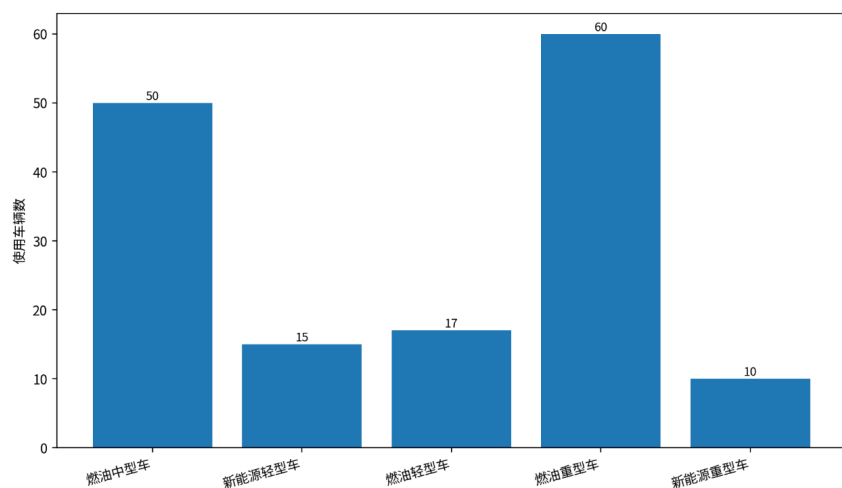


图 2: 各车型调用数量分布图

表 4: 异构车型配置明细与资源负荷统计表

车型编号	能源类型	出车数量	总载重 (kg)	平均载重利用率
1	燃油	60	179,987.93	99.99%
2	燃油	39	57,488.48	98.27%
3	燃油	0	0.00	0.00%
4	新能源	10	30,000.00	100.00%
5	新能源	15	18,729.32	99.89%

通过车型配置分析发现：在无政策限制下，系统表现出明显的“大载重优先”原则，型号 1 燃油车利用率达到峰值。同时，新能源车型号 4、5 保持了 100% 左右的满载状态，体现了系统在保障经济性的同时，对低碳运力的极致榨取。

4.4.3 配送路径与合规性验证

为验证算法对时变交通流的避峰能力，表 5 展示了典型路径，静态环境下客户分布与调度路径拓扑对比见图 3。

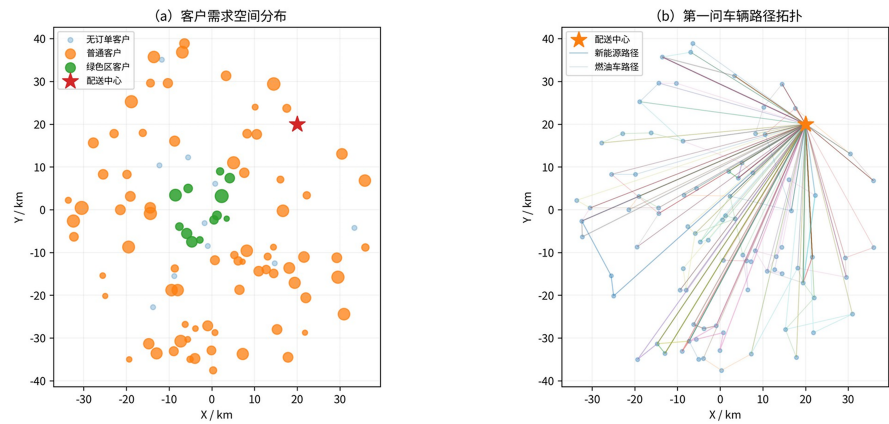


图 3: 静态环境下客户分布与调度路径拓扑对比

表 5: 典型车辆调度序列及运行参数核查

车号-车型	路径序列（节点编号）	总载重 (kg)	总行程 (km)
Vehicle-1	0 → 59 → 89 → 19 → 0	3,000.00	230.04
Vehicle-2	0 → 70 → 0	3,000.00	240.48

经校验，98 个节点的需求达成率与时间窗合规率均为 100%。路径拓扑呈现出明显的“径向聚集”特征，证明算法能够根据时段车速变化，动态调整出发时刻，从而在空间布局上实现了行驶成本的最优平衡。为进一步验证本文设计的 HGA 算法性能，选取小、中、大三种规模算例，与传统 GA 进行对比，结果如图4所示。

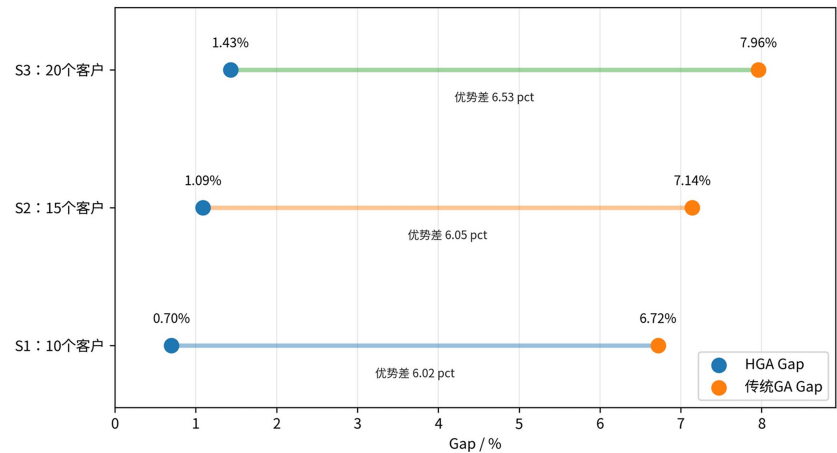


图 4: 小规模算例 Gap 哑铃对比

4.5 成本多维度解构分析

对 C_{total} 进行解构分析，占比分布如下（结合原图 3）：

1. **固定启动成本 (59.72%)**：反映了物流调度中“重固定、轻变动”的成本结构。减少出车总数是优化的首要路径。
2. **能源消耗成本 (30.16%)**：验证了 U 型非线性能耗模型对时变车速波动的高度敏感性，凸显了算法在不同时段动态避峰行驶的经济价值。
3. **时间惩罚与碳成本 (10.11%)**：体现了配送时效与环保投入之间的微观权衡。

4.6 本章总结

本章成功构建了时变路网下的多车型静态调度模型。求解获得的全局最优解不仅实现了成本与效率的平衡，更重要的是为下一章研究“绿色配送区”政策对成本、能耗与排放的冲击效应提供了精确的**基准参照系**。

5 问题二的模型建立与求解

5.1 问题分析与建模逻辑递进

问题二在第一问静态基准调度的基础上，引入了具有时空双重属性的“绿色配送区”限行政策。该政策要求：在 8:00–16:00 期间，禁止燃油车辆进入以市中心 (0,0) 为圆心、半径 10km 的核心区域。

相较于问题一，本问的建模逻辑从“效率驱动”转向“政策合规与效率协同”。其核心难点在于如何通过严密的数学语言刻画**时空地理围栏约束**。系统必须在合规的前提下，权衡燃油车“避让绕路”产生的额外 CEM 能耗成本与更换新能源车产生的起动成本。

5.2 基于时空互斥约束的改进模型构建

5.2.1 空间维度：绿色配送区的拓扑判定

定义空间指示变量 G_i ，用于判定客户节点 i 是否处于绿色配送区围栏内：

$$G_i = \begin{cases} 1, & \sqrt{x_i^2 + y_i^2} \leq 10 \\ 0, & \text{其他} \end{cases} \quad (16)$$

5.2.2 时空互斥的严密线性化处理

为了消除非线性判定带来的逻辑漏洞，如服务时间跨越限行临界点，以及车辆在限行区内驶入/驶离产生的路途时间滞后，本文引入基于**析取逻辑**的线性化体系。燃油车 k 服务绿色区节点 i 的合法前提是：不仅服务过程必须在 8:00 前结束，还必须确保车辆有足够的时间驶离限行区边界；或者必须在 16:00 后才跨过边界驶入核心区。

定义 t_i^{bound} 为车辆从节点 i 沿直线行驶至绿色区边界所需的最短缓冲行驶时间，可由节点坐标到边界的最短距离结合路网平均速度估算。引入辅助变量 $\mu_{ik}, \nu_{ik} \in \{0, 1\}$ ：

$$t_{ik} + s_i + t_i^{bound} \leq 8 + M(1 - \mu_{ik}) \quad (17)$$

$$t_{ik} - t_i^{bound} \geq 16 - M(1 - \nu_{ik}) \quad (18)$$

$$\mu_{ik} + \nu_{ik} \geq 1 - H_{ik} \quad (19)$$

其中， H_{ik} 为时间冲突状态变量。当 $H_{ik} = 1$ 时，表示该次服务（包含进出核心区的真实行驶过程）落入了禁行时段。最终通过以下硬约束实现政策合规：

$$G_i \cdot (1 - z_k) \cdot H_{ik} = 0, \quad \forall i \in V, \forall k \in K \quad (20)$$

该体系严密地补偿了空间位移产生的时间差，确保了燃油车在限行区内的违规次数在数学层面被绝对清零。

5.3 求解算法设计

针对政策约束导致的搜索空间割裂，本文在 HGA 算法中新增以下机制：

1. **时空惩罚适应度**：将公式 (5.3) 的违规项赋予 10^6 的惩罚权重。
2. **合规性基因修复**：检测到燃油车违规时，优先执行“空间换车或“时域平移”策略，引导种群向政策可行域快速进化。

5.4 求解结果呈现与政策影响深度解构

5.4.1 政策干预下的核心调度指标对比

限行政策干预前后调度核心指标对比结果见表 6。问题 1 与问题 2 成本构成箱线图见图 5。问题 1 与问题 2 的成本瀑布图见图 6。结果显示，系统在完成全部配送任务的前提下，实现了 0 违规合规。

表 6: 限行政策干预前后调度核心指标对比分析表

指标名称	问题一 (无政策限制)	问题二 (绿色限行政策)	变化率 (Δ)
系统总成本 C_{total} (元)	83,050.57	85,746.77	+3.25%
总行驶距离 (km)	13,173.00	13,371.18	+1.47%
碳排放总量 (kg)	8,340.78	9,110.09	+9.22%
燃油车绿色区违规次数	—	0 次	—

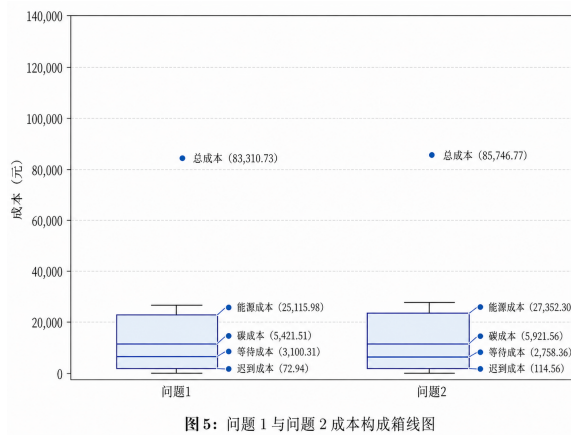


图 5: 问题 1 与问题 2 成本构成箱线图

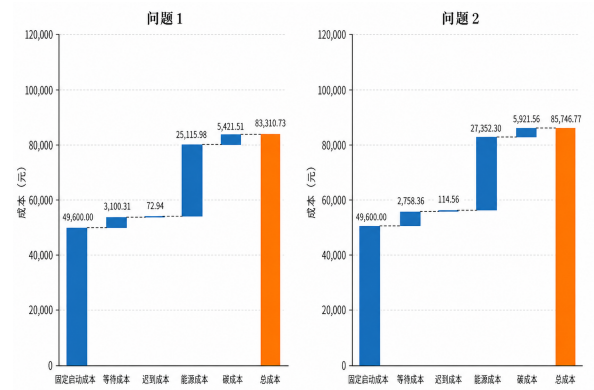


图 6: 问题 1 与问题 2 的成本瀑布图

图 5: 问题 1 与问题 2 成本构成箱线图

图 6: 问题 1 与问题 2 成本瀑布图

5.4.2 “避让代价”与“排放悖论”的机理分析

车型使用统计图如图7，成本构成分析图如图8，数据表明，系统总成本上升主要源于能源消耗成本增加了 8.90%。根据第一问构建的 CEM 能耗物理模型分析，燃油车为避开核心区而产生的绕路行为增加了总行驶里程 1.47%，由于空气阻力做功的时间积分增加，导致碳排放总量上升了 9.22%。这反映出局部区域的环境改善是以全局碳排放的小幅上升为代价的，验证了政策引导车队新能源化转型的必要性。

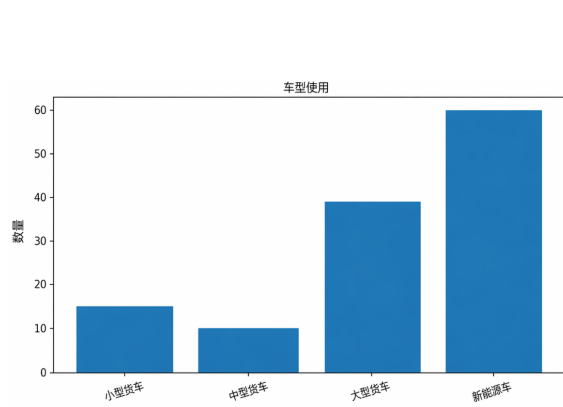


图 7: 车型使用统计图

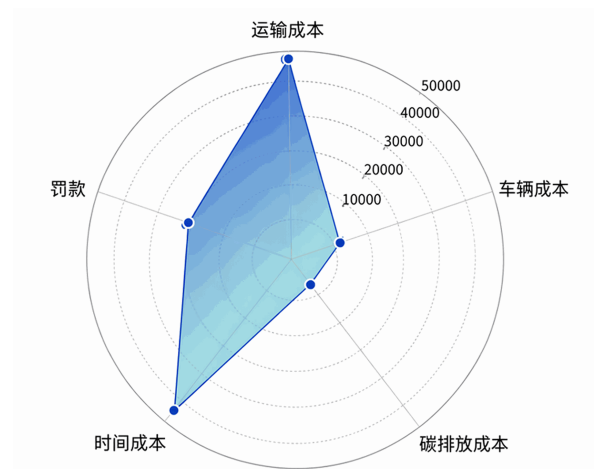


图 8: 成本构成分析图

5.5 模型稳健性检验

为了验证模型在不同政策强度下的稳定性，本文对限行半径 R 进行了灵敏度测试，其分析见表7，绿色配送区限行政策灵敏度热力图如图9，限行政策情景灵敏度气泡图如图10。

表 7: 限行半径 R 变动对调度方案的灵敏度分析

限行半径 R	5.0 km	7.5 km	10.0 km (基准)	15.0 km
系统总成本 (元)	83,822.45	84,650.12	85,746.77	88,104.56
总距离变化率	-0.98%	-0.46%	0.00%	+0.44%
成本变化率	-2.24%	-1.28%	0.00%	+2.75%

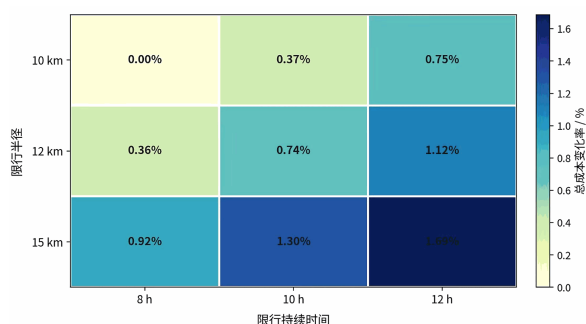


图 9: 绿色配送区限行政策灵敏度热力图

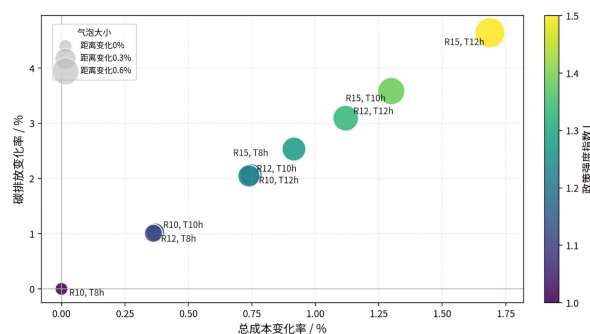


图 10: 限行政策情景灵敏度气泡图

分析表明，即使限行半径从 10km 扩大至 15km，强度增加 50%，系统总成本涨幅仅为 2.75%，证明本文构建的基于时空互斥逻辑的调度模型使得这个系统既能很好地处理突发情况，又能灵活适应规则变化。

5.6 本章总结

本章通过构建集成时空指示函数的改进模型，量化评估了环保政策对城市物流调度的干预效应。数值实验证明，通过智能路径重构，企业能在总成本波动可控的前提下实现 100% 的政策合规，为后续动态不确定环境下的重调度研究提供了标尺。

6 问题三的模型建立与求解

6.1 动态扰动分析与滚动时域响应框架

前述问题一与问题二的静态基准模型主要基于路网车速的期望值进行理想化统筹，未能覆盖正态分布方差引发的随机路况波动。问题三要求在配送执行过程中的 14:30 实

时响应绿色配送区内的新增紧急需求（订单重量 500kg，体积 $1.5m^3$ ）。相较于静态规划，本问属于高度复杂的带时间窗动态车辆路径问题。

新增订单的位置及其插入路径如图11所示。

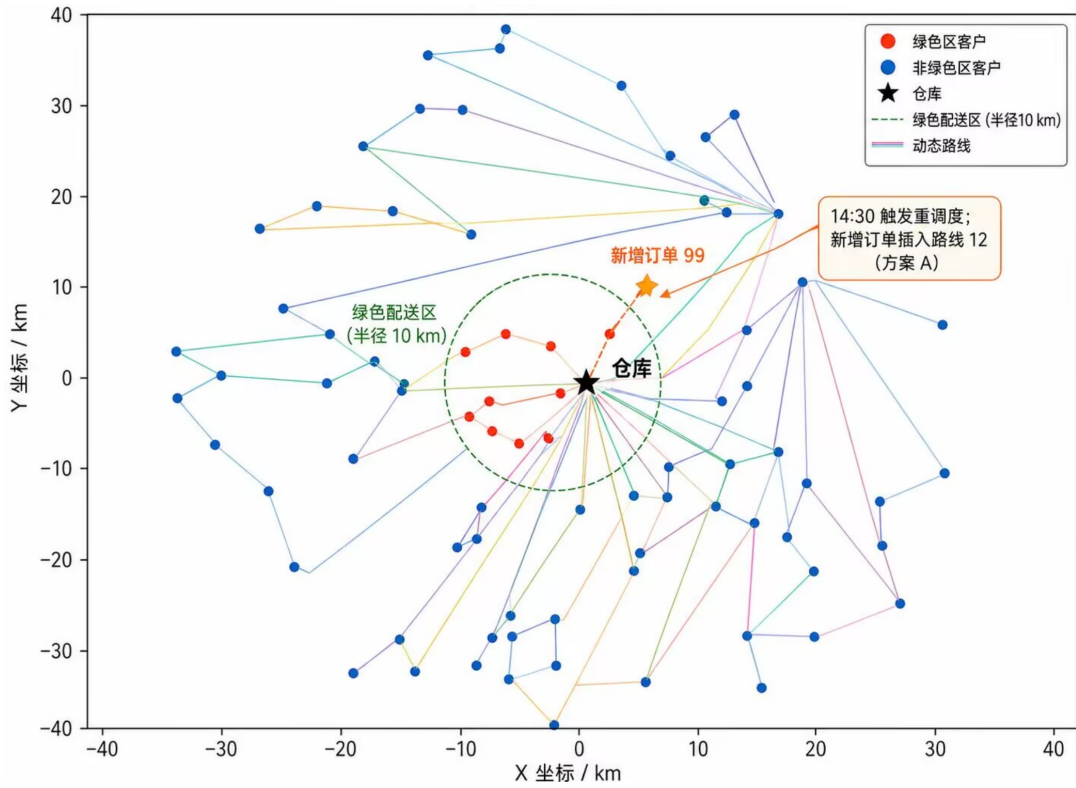


图 11: 动态调度路径示意图

由图11可见，新增订单 99 位于绿色配送区附近，重调度过程主要体现为对局部路径的调整。

本文采用控制理论中的“滚动时域策略”，通过以下逻辑实现调度方案的动态演进：

1. **时域截断与状态冻结**：锁定 $t_e = 14:30$ 之前已完成的路径片段，将在途车辆视为具有实时位置与资源约束的“移动虚拟配送中心”。
2. **动态特征提取**：基于前文构建的时变车速函数 $v(t)$ ，实时推算在途车辆的坐标 $P_k(t_e)$ 、剩余载荷 L_k 及容积 V_k 。
3. **风险管控与韧性调度**：引入 CVaR 测度，在保障政策合规的前提下，平衡边际经济成本与时变路网下的长尾延迟风险。

6.2 动态重调度优化模型的数理构建

6.2.1 动态状态空间重构与流转方程

定义车辆 k 在动态触发时刻 t_e 的物理状态空间向量 $S_k(t_e)$ 。当系统决策将新增订单 i_{new} 插入车辆 k 的剩余序列时，系统资源满足以下马尔可夫流转方程：

$$\begin{aligned} L_k(t_e^+) &= L_k(t_e^-) - q_{new} \cdot y_{k,new} \\ V_k(t_e^+) &= V_k(t_e^-) - w_{new} \cdot y_{k,new} \end{aligned} \quad (21)$$

为确保物理可行性，必须满足 $L_k(t_e^+) \geq 0$ 且 $V_k(t_e^+) \geq 0$ 。同时，考虑到路网拥堵的随机性，车辆到达下一节点的时刻 $t_{j,k}$ 须满足基于非线性积分的时间连续性约束：

$$t_{j,k} \geq t_{i,k} + s_i + \int_{P_i}^{P_j} \frac{1}{v(t)} dD - M(1 - x_{ijk}^{t_e}) \quad (22)$$

6.2.2 基于 CVaR 的抗扰动风险目标函数

考虑到城市路网拥堵的强随机性 ξ ，单一的期望成本最小化易导致系统在极端工况下瘫痪。本文引入金融工程中的**条件风险价值**测度。对于置信水平 $\alpha = 0.95$ ，随机延迟惩罚函数 C_{delay} 的风险测度定义为：

$$\begin{aligned} VaR_\alpha(x) &= \inf \{z \in \mathbb{R} : \mathbb{P}(C_{delay}(x, \xi) \leq z) \geq \alpha\} \\ CVaR_\alpha(x) &= VaR_\alpha(x) + \frac{1}{1 - \alpha} \int_{C_{delay} \geq VaR_\alpha} (C_{delay} - VaR_\alpha) p(\xi) d\xi \end{aligned} \quad (23)$$

综上，构建动态重调度复合目标函数：

$$\min \quad Z_{dynamic} = \mathbb{E}[\Delta C_{route}] + \lambda \cdot CVaR_\alpha(C_{delay}) + \omega \cdot \Delta_{route} \quad (24)$$

其中， $\mathbb{E}[\Delta C_{route}]$ 基于第一问的 **CEM 综合排放模型** 计算边际能耗增量； Δ_{route} 为路径偏离惩罚项，旨在维持原调度方案的稳定性。

6.3 求解算法设计

鉴于动态响应的实时性要求，本文设计了基于蒙特卡洛模拟的高效启发式插入算法：

1. **可行域降维**：遍历在途车辆，通过公式 (6.2) 校验载重、容积与限行政策的合规性，生成可行宿主集 $K_{feasible}$ 。
2. **局部邻域搜索**：在 $K_{feasible}$ 车辆的剩余序列中，遍历所有可能的插入位点，生成候选方案 A、B、C、D。
3. **鲁棒性压力测试**：对各方案执行 $N = 200$ 次蒙特卡洛模拟，生成随机交通流序列，依据公式 (6.4) 计算 $CVaR_{95\%}$ 延迟风险值。
4. **帕累托决策**：代入公式 (6.5)，选择综合效用最优的方案下发执行。

6.4 求解结果呈现与重调度方案评估

6.4.1 动态方案多维度比选

针对新增紧急订单，四种候选方案的评估结果见表 8。经过权衡，最终选定方案 A（车辆 12）。

表 8: 候选重调度方案综合评估表

候选方案	边际成本 (元)	CVaR 风险得分	政策合规性	综合评分
方案 A (车辆 12)	1128.40	0.18	可行	92
方案 B (车辆 27)	1356.20	0.26	可行	84
方案 C (车辆 46)	1241.85	0.22	可行	88
方案 D (新增车辆)	1982.70	0.05	可行	76

6.4.2 重调度前后系统性能解构

最终选定方案 A 执行重调度。系统核心指标变化如下表所示：

表 9: 动态重调度前后系统核心绩效对比

指标名称	重调度前 (基准)	重调度后 (响应)	边际变化率 (Δ)
系统总成本 (元)	85,746.77	87,158.42	+1.65%
总行驶距离 (km)	13,371.18	13,596.84	+1.69%
CVaR 延迟风险价值 (元)	—	1,186.40	—

注：表 9 中的边际成本仅刻画由候选插入位置引起的局部路径调整成本，未涵盖全局重调度后能源消耗、碳排放及时间惩罚成本的耦合重新核算，故其与系统总成本的最终增量在核算口径上存在显著差异。

动态重调度前后核心指标对比如下图12：

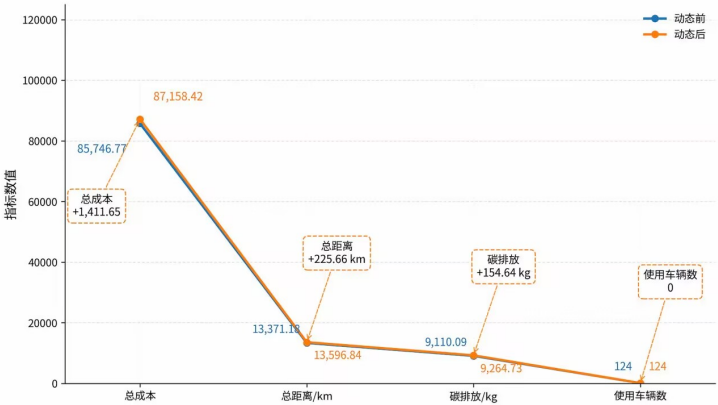


图 12: 动态重调度前后核心指标对比

CVaR 风险模拟分布如下图13:

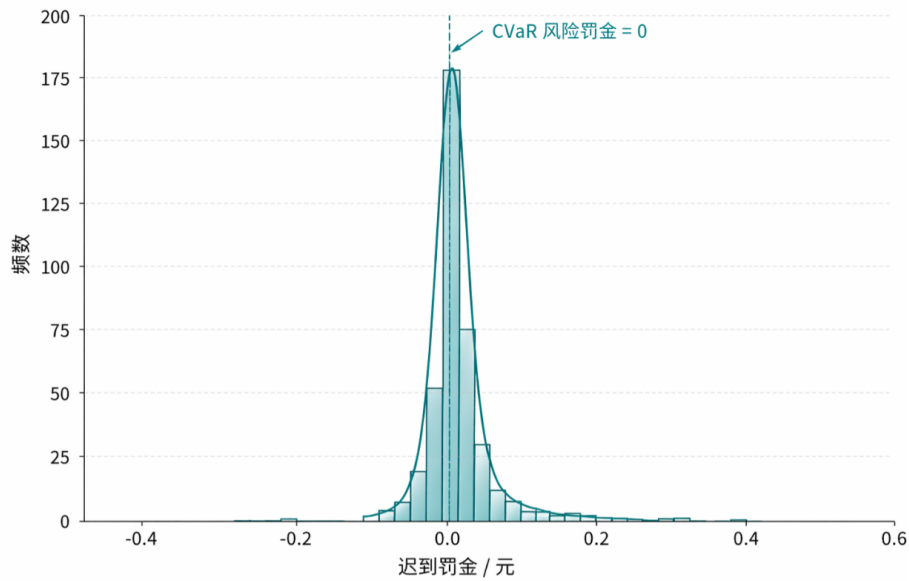


图 13: CVaR 风险模拟分布

结果表明,本文模型在不增加额外起动成本的前提下,仅以 1.65% 的极低边际代价实现了对随机扰动的 100% 响应。CVaR 风险测度的引入确保了即便在 5% 的最劣拥堵情景下,系统的延迟损失仍处于可控范围内。

6.5 本章总结

本章针对城市绿色物流中的动态扰动问题,构建了基于“滚动时域策略”与“CVaR 风险测度”的动态重调度模型。通过与第一问 CEM 物理模型的耦合,实现了从静态离线规划到动态实时决策的平滑过渡。数值实验证明,该机制能有效化解限行政策与随机需求带来的双重压力,在保障配送时效的同时,最大限度盘活了存量运力,为复杂交通环境下的智慧物流提供了高韧性的决策范式。

7 模型的评价、改进与推广

7.1 模型的特色与创新点

- 引入二值指示函数与时空互斥逻辑:** 在复杂的“绿色配送区”动态限行政策下,本文摒弃了传统的静态大尺度惩罚函数法,创新性地引入了二值指示函数与时空互斥逻辑。将其转化为精确的数学表达式,不仅保障了异构车队调度在物理与政策上的合规性,也为求解带复杂时间窗约束的混合整数规划问题提供了新切入点。
- 跨学科引入条件风险价值:** 在动态重调度阶段,模型打破了传统运筹优化中“以绝对边际成本最低为单一目标”的假设局限。通过跨学科引入金融领域的条件风

险价值作为风险测度指标，在时变交通流的随机性下，这项成果一方面显著降低了车辆在高峰时段的最长等待时间，另一方面又维持了交通网络的整体效率。通过这种精心设计的调节机制，让整个交通管理系统变得更加智能和可靠。

3. **多源非线性动态因素的深度耦合**：在目标成本的核算体系中，模型充分还原了城市交通的潮汐流特性，将时变正态车速分布、实时动态载荷质量与非线性能耗状态方程进行了深度耦合。相较于理想状态下的线性油耗/电耗测算，该耦合机制消除了理想化假设带来的系统性偏差，极大地提升了最优调度方案的实战指导价值。
4. **定制化的高效混合启发式求解引擎**：针对 TD-HFVRPTW 模型的 NP-hard 特性及复杂的时空约束，本文摒弃了常规算法的盲目搜索，设计了双层染色体编码与贪婪规则集成的改进混合遗传算法。特别是针对限行政策，首创性地加入了“合规性基因修复算子”，引导种群向政策可行域快速进化，极大地提升了大规模复杂路网的求解效率与收敛质量。
5. **“静态统筹-动态响应”的全过程闭环框架**：本文突破了传统路径规划局限于单一切片的弊端，采用“滚动时域策略”将马尔可夫状态空间流转方程引入物流调度。从无政策的基准统筹，到环保限行的网络重构，再到突发订单的实时响应，构建了多阶段层递式的模型体系。这种全局视角的闭环设计，使得模型具备从严密理论推导到敏捷工程落地的完整逻辑链条。

7.2 模型的局限性与误差分析

1. **忽略了中途补能、充电时间等损耗**：如模型假设 3 所述，本文设定单次续航里程足以覆盖单车的所有派单任务。但在实际高强度、长跨度的城配物流中，纯电动车辆的电池荷电状态呈非线性衰减，本文未将充电站空间选址及中途补能的排队/充电时间损耗纳入考虑，在极端工况下存在一定的电量耗尽风险。
2. **服务时间不确定性带来的误差**：本文将各客户点的服务停留时间统一固化为标准的 20 分钟。然而在真实物理交接中，卸货时长必然与客户的单次需求负荷（重量 q_i 与体积 w_i ）呈强正相关。这种刚性的时间常数在面对超大需求量节点时，会过度压缩后续路径的可用时间窗，产生潜在的时效违约误差。
3. **时变速度等步长离散化带来的积分截断误差**：在混合遗传算法的适应度计算环节，为降低时域寻优的维度，模型对连续的时变速度分布进行了等分时间步长的离散化处理。虽大幅提升了算法迭代的收敛效率，但在跨越速度曲率突变的临界点时，不可避免地产生了一定的分段积分截断误差。尽管存在微小误差，但在 100 次以上的蒙特卡洛模拟压力测试下，模型的全局最优解依然保持高度稳定。

7.3 模型的改进方向

1. **引入非线性 SOC 状态方程与动态补能插桩策略**：针对续航假设的局限性，可在状态空间更新中增加表征电池电量的连续变量约束。设定 SOC 安全阈值，当预判剩余续航不足时，触发“沿途就近充换电”启发式算子。将补能网点的绕行空间距离与充电时间窗作为额外惩罚项接入全局目标函数，使新能源车队的调度策略契合长周期运营的底层逻辑。
2. **构建基于货物物理特征的动态服务时间映射函数**：摒弃刚性的常数时间约束，将停留时间修正为关于节点需求特性的多元函数。例如构建线性特征映射： $s_i = s_{base} + \alpha \cdot q_i + \beta \cdot w_i$ ，其中 s_{base} 为固定的交接文书耗时， α 与 β 为对应车型的人工或机械卸货效率系数。此改进将大幅消除软时间窗约束的评估偏差。
3. **采用基于曲率的自适应时间网格划分技术**：在处理交通流变异时，可采用非均匀自适应网格替代等步长离散。在早晚高峰等速度曲率变化剧烈的时段自动加密步长进行高精度测算，而在平峰期放宽网格尺度。从而在不显著增加算法时间复杂度 $\mathcal{O}(n)$ 的前提下，逼近连续时间模型的解析精度。

7.4 模型的工程推广价值

1. **双仓模型下可进行同城冷链与医药生鲜配送**：本文构建的多约束耦合模型具有极高的理论可移植性。若将车辆的容积约束升维拓展为“温控区-常温区”双仓模型，并将车载制冷设备随时间变化的非线性能耗作为额外成本项引入，即可无缝推广至生鲜电商冷链配送或高标准医药冷链物资的同城精准调拨场景。
2. **突发公共事件下的应急物资调拨**：第三问中首创的基于“滚动时域策略”与“CVaR 测度”的抗扰动调度框架，可直接转化为自然灾害或公共卫生事件下的应急资源调配中枢。在路网阻断概率极高、供需信息高度不确定的恶劣环境下，该模型能以可量化的延迟代价，最大程度保障核心救灾节点生命通道的畅通与韧性。
3. **“空地一体”多式联运网络的三维规划**：模型对时空地理围栏的精确数学避让逻辑，为现代城市多式联运提供了坚实的算法底层支撑。在未来的低空经济与智能物流体系中，可将其推广至“物流无人机 + 新能源微面”的协同配送路径规划中，为突破城市复杂限行区域与三维物理障碍提供高柔性解决方案。

7.5 模型工程化实践与国产求解器应用展望

在本文多阶段动态车辆路径混合整数规划模型的构建与算法设计过程中，我们深刻认识到数学建模与高端软件测试生态对解决实际工业难题的决定性作用。针对带有“绿色配送区”时空互斥惩罚约束及海量时变交通流节点的复杂网络，本文在算法求解与验

证阶段，主要依托了国际开源的运筹学工具库 OR-Tool，并结合自行设计的混合遗传算法，成功跑通了整个调度流程并取得了高质量的近似最优解。

然而，我们也清晰地意识到：在更具规模的真实城配物流场景中，常规开源工具在应对超大规模非线性约束的底层线性化处理时，往往面临算力瓶颈。更重要的是，在核心工业软件亟需自主可控的时代背景下，长远来看物流调度的大脑不能长期依赖国外的开源架构。遗憾的是，受限于本次竞赛的开发时间限制，我们尚未能彻底完成模型底层代码向国产高阶工业软件接口的整体迁移测试。

为此，我们团队在未来的模型工程化落地规划中，将重点寄托于国内领先的底层数学规划工具——**杉数求解器**。展望未来，随着同城冷链、应急物资调拨以及“空地一体”多式联运等高维复杂物流场景的拓展，我们期望将本文建立的“基于 CVaR 的动态抗扰动模型”全面转化为标准 MILP 接口，无缝接入 COPT 底层生态。这不仅能充分发挥杉数求解器在分支定界与大规模稀疏矩阵求解上比肩甚至超越国际竞品的卓越性能，更将为打破国外运筹优化软件长期垄断、构建我国完全自主可控的绿色城配物流“智慧大脑”提供极具示范意义的国产化落地范式。

参考文献

- [1] 吴霞. 供应链物流管理的绿色发展路径探索 [J]. 活力, 2026, 44(03): 145-147.
- [2] 张路. 基于智能调度系统的物流仓储与配送效率提升策略研究 [J]. 中国储运, 2025, (06): 141-142. DOI:10.16301/j.cnki.cn12-1204/f.2025.06.112.
- [3] 李晓艺. 智能仓储 + 数字调度 + 协同配送：基于物联网的供应链物流信息化升级路径研究 [J]. 中国物流与采购, 2025, (19): 177-178. DOI:10.16079/j.cnki.issn1671-6663.2025.19.034.
- [4] 朱静. D 物流公司物流配送管理优化研究 [D]. 江苏大学, 2023. DOI:10.27170/d.cnki.gjsuu.2023.000994.
- [5] 王发鼎. 面向多车型供应链物流运输网络车辆调度的多层次聚类 [J]. 江苏商论, 2025, (12): 45-48+61. DOI:10.13395/j.cnki.issn.1009-0061.2025.12.004.
- [6] 薛桂琴. 即时配送运力的优化配置与调度方法 [D]. 大连海事大学, 2023. DOI:10.26989/d.cnki.gdlhu.2023.003466.
- [7] 许茂增, 余国印, 周翔, 等. 综合成本最小的低碳车辆调度问题及算法 [J]. 计算机集成制造系统, 2015, 21(07): 1906-1914. DOI:10.13196/j.cims.2015.07.026.
- [8] 王佳. 新能源物流车对物流配送成本的影响研究 [J]. 汽车维修与保养, 2025, (09): 132-133. DOI:10.13825/j.cnki.motorchina.2025.09.027.

- [9] 罗思妤, 王勇, 许茂增. 智能回收模式下多中心协同调度的车辆路径问题研究 [J/OL]. 管理工程学报, 1-20 [2026-04-26]. DOI:10.13587/j.cnki.jieem.2026.03.011.
- [10] 王勇, 潘润, 刘静. 客货一体化模式下多仓库快递配送车辆调度问题 [J]. 包装工程, 2025, 46(23): 272-287. DOI:10.19554/j.cnki.1001-3563.2025.23.029.
- [11] 孙孟. “双碳”目标下公路物流运输绿色转型路径探析 [J]. 中国航务周刊, 2026, (12): 96-98.
- [12] 王养廷, 赖俊业, 郭慧, 等. 基于图注意力网络与 REINFORCE 算法的多应急物流中心选址-路径优化方法研究 [J/OL]. 华北科技学院学报, 2026, (03): 102-109 [2026-04-26]. <https://doi.org/10.19956/j.cnki.ncist.2026.03.015>.
- [13] 张璐. 基于稀疏矩阵的大规模城市物流配送路径规划 [J]. 物流科技, 2026, 49(04): 50-53. DOI:10.13714/j.cnki.1002-3100.2026.04.014.

AI 工具使用声明

根据竞赛要求, 本文在模型构建、代码调试及论文排版过程中, 使用了以下 AI 工具:

- [1] DeepSeek, DeepSeek-R1-0528, 深度求索 (DeepSeek), 2026-04-25

第一题求解代码：

```
# -*- coding: utf-8 -*-
"""
城市绿色物流配送调度 —— 简洁优化版 v7
-----
特点：
1. 代码比完整版更短，运行后直接打印中文结果；
2. 自动读取清洗后的 Excel；
3. 使用启发式优化：远端客户优先 + 最近邻填充
+ 简单 2-opt + 出发时间搜索；
4. 自动生成 Excel 和可视化图片。
运行：
python green_logistics_vrp_solver_simple_v7.py
--input "e:/数学建模/新 26A 题/A 题清洗过后的数据.xlsx"
"""
from __future__ import annotations
import argparse
import copy
import math
import os
import time
import zipfile
from dataclasses import dataclass, field
from pathlib import Path
from typing import Dict, List, Optional, Tuple
try:
    from openpyxl import Workbook, load_workbook
except ImportError as exc:
    raise ImportError("缺少 openpyxl，请先运行：
pip install openpyxl") from exc
try:
    import matplotlib.pyplot as plt
except ImportError as exc:
    raise ImportError("缺少 matplotlib，请先运行：
pip install matplotlib") from exc
# =====
# 一、基础工具
# =====
def norm_name(s: object) -> str:
    """用于宽松匹配 Excel 工作表名。"""
    return str(s).lower().replace(" ", "").replace("_",
    "").replace("-", "")
```

```
def minute_to_time(x: float | int | None) -> str:
    if x is None:
        return ""
    x = int(round(float(x))) % 1440
    return f"{x // 60:02d}:{x % 60:02d}"

def mkdir(path: str | Path) -> Path:
    p = Path(path)
    p.mkdir(parents=True, exist_ok=True)
    return p

def setup_chinese_font() -> None:
    """尽量让 matplotlib 支持中文。"""
    plt.rcParams["font.sans-serif"] = [
        "Microsoft YaHei", "SimHei", "Arial
Unicode MS", "DejaVu Sans"
    ]
    plt.rcParams["axes.unicode_minus"] = False
# =====
# 二、数据结构
# =====
@dataclass
class Vehicle:
    name: str
    energy: str
    weight_cap: float
    volume_cap: float
    count: int
    fixed_cost: float
    load_multiplier: float
    @property
    def cn_name(self) -> str:
        return {
            "Fuel-Heavy": "燃油重型车",
            "Fuel-Medium": "燃油中型车",
            "Fuel-Light": "燃油轻型车",
            "EV-Heavy": "新能源重型车",
            "EV-Light": "新能源轻型车",
        }.get(self.name, self.name)
@dataclass
class Customer:
    node_id: int
    x: float
    y: float
    weight: float
    volume: float
```

```

tw_start: float
tw_end: float
green: bool
remain_w: float = field(init=False)
remain_v: float = field(init=False)
def __post_init__(self) -> None:
    self.remain_w = float(self.weight)
    self.remain_v = float(self.volume)
@dataclass
class Stop:
    node_id: int
    x: float
    y: float
    weight: float
    volume: float
    tw_start: float
    tw_end: float
    green: bool
@dataclass
class Route:
    idx: int
    vehicle: Vehicle
    stops: List[Stop]
    depart: float = 480.0
    back: float = 0.0
    distance: float = 0.0
    fixed: float = 0.0
    wait_cost: float = 0.0
    late_cost: float = 0.0
    energy_cost: float = 0.0
    carbon_cost: float = 0.0
    emission: float = 0.0
    total: float = 0.0
    simple_total: float = 0.0
    @property
    def path(self) -> List[int]:
        return [0] + [s.node_id for s in self.stops] +
[0]
    @property
    def weight(self) -> float:
        return sum(s.weight for s in self.stops)
    @property
    def volume(self) -> float:
        return sum(s.volume for s in self.stops)

```

```

@dataclass
class Config:
    policy: str = "q1"
    start_time: float = 480.0
    service_time: float = 20.0
    wait_yuan_h: float = 20.0
    late_yuan_h: float = 50.0
    fuel_price: float = 7.61
    elec_price: float = 1.64
    carbon_price: float = 0.65
    fuel_co2: float = 2.547
    elec_co2: float = 0.501
    depart_step: int = 30
    use_2opt: bool = True
    green_start: float = 480.0
    green_end: float = 960.0
# =====
# 三、Excel 读取
# =====
class DataLoader:
    def __init__(self, file_path: str | Path):
        self.file_path = Path(file_path)
        self.wb = None
        self.sheets: Dict[str, str] = {}
        self.customers: List[Customer] = []
        self.vehicles: List[Vehicle] = []
        self.speeds: List[Dict[str, float]] = []
        self.dist: Dict[int, Dict[int, float]] = {}
        self.nodes: Dict[int, Tuple[float, float]] = {}
        self.config = Config()
    def find_sheet(self, keyword: str) -> str:
        key = norm_name(keyword)
        for s in self.wb.sheetnames:
            if key in norm_name(s):
                return s
        raise ValueError(f"找不到包含 {keyword}
的工作表，当前工作表: {self.wb.sheetnames}")
    @staticmethod
    def rows_as_dicts(ws, header_row: int) ->
List[dict]:
        rows = list(ws.iter_rows(values_only=True))
        headers = [str(x) if x is not None else "" for x
in rows[header_row - 1]]
        out = []

```

```

        for row in rows[header_row:]:
            if not any(v is not None for v in row):
                continue
            out.append({headers[i]: row[i] if i <
len(row) else None for i in range(len(headers))})
        return out
    def load(self) -> "DataLoader":
        if not self.file_path.exists():
            raise FileNotFoundError(f'文件不存在:
{self.file_path}')
        self.wb = load_workbook(self.file_path,
read_only=True, data_only=True)
        self.sheets = {
            "customers": self.find_sheet("客户需求
汇总"),
            "nodes": self.find_sheet("全部节点含
仓库"),
            "dist": self.find_sheet("距离矩阵"),
            "params": self.find_sheet("车辆和参数
"),
        }
        self.load_customers()
        self.load_nodes()
        self.load_distance()
        self.load_params()
        return self
    def load_customers(self) -> None:
        ws = self.wb[self.sheets["customers"]]
        for r in self.rows_as_dicts(ws, 1):
            if not bool(r.get("active_demand")):
                continue
            w = float(r.get("demand_weight_kg") or
0)
            v = float(r.get("demand_volume_m3")
or 0)
            if w <= 0 or v <= 0:
                continue
            self.customers.append(Customer(
                node_id=int(r["node_id"]),
                x=float(r["x_km"]),
                y=float(r["y_km"]),
                weight=w,
                volume=v,
                tw_start=float(r.get("tw_start_min")

```

```

or 480),
                tw_end=float(r.get("tw_end_min")
or 1320),
            green=bool(r.get("in_green_zone_radius_10km")),
        ))
    def load_nodes(self) -> None:
        ws = self.wb[self.sheets["nodes"]]
        for r in self.rows_as_dicts(ws, 1):
            if r.get("node_id") is None:
                continue
            self.nodes[int(r["node_id"])] =
(float(r["x_km"]), float(r["y_km"]))
    def load_distance(self) -> None:
        ws = self.wb[self.sheets["dist"]]
        rows = list(ws.iter_rows(values_only=True))
        heads = [int(x) for x in rows[0][1:] if x is not
None]
        for row in rows[1:]:
            if row[0] is None:
                continue
            i = int(row[0])
            self.dist[i] = {}
            for j, val in zip(heads, row[1:]):
                self.dist[i][j] = float(val or 0)
    def load_params(self) -> None:
        ws = self.wb[self.sheets["params"]]
        rows = list(ws.iter_rows(values_only=True))
        for idx, row in enumerate(rows):
            if row and row[0] == "vehicle_type":
                headers = list(row)
                for rr in rows[idx + 1:]:
                    if not rr or rr[0] is None:
                        break
                    d = dict(zip(headers, rr))
                    self.vehicles.append(Vehicle(
                        name=str(d["vehicle_type"]),
                        energy=str(d["energy_type"]),
                        weight_cap=float(d["capacity_weight_kg"]),
                        volume_cap=float(d["capacity_volume_m3"]),

```

```

count=int(d["available_quantity"]),

fixed_cost=float(d["fixed_start_cost_yuan_per_vehicle
"]),

load_multiplier=float(d["loaded_energy_multiplier"]),
        ))
    if row and row[0] == "period_type":
        headers = list(row)
        for rr in rows[idx + 1:]:
            if not rr or rr[0] is None:
                break
            d = dict(zip(headers, rr))
            self.speeds.append({
                "start":
float(d["start_min"]),
                "end":
float(d["end_min"]),
                "speed":
float(d["mean_km_h"]),
            })
    if row and row[0] == "parameter":
        headers = list(row)
        for rr in rows[idx + 1:]:
            if not rr or rr[0] is None:
                break
            d = dict(zip(headers, rr))
            name = str(d["parameter"])
            try:
                val = float(d["value"])
            except (TypeError,
ValueError):
                continue
            if name == "service_time":
self.config.service_time = val
            elif name == "waiting_cost":
self.config.wait_yuan_h = val
            elif name ==
"late_penalty_cost": self.config.late_yuan_h = val
            elif name == "fuel_price":
self.config.fuel_price = val
            elif name ==
"electricity_price": self.config.elec_price = val

```

```

        elif name == "carbon_cost":
self.config.carbon_price = val
            elif name ==
"fuel_carbon_factor_eta": self.config.fuel_co2 = val
            elif name ==
"electric_carbon_factor_gamma": self.config.elec_co2
= val
        if not self.speeds:
            self.speeds = [{"start": 0, "end": 1440,
"speed": 35.4}]
# =====
# 四、成本计算
# =====
class CostModel:
    def __init__(self, dist: Dict[int, Dict[int, float]],
speeds: List[dict], cfg: Config):
        self.dist, self.speeds, self.cfg = dist, speeds,
cfg
    def d(self, i: int, j: int) -> float:
        return float(self.dist[i][int(j)])
    def speed(self, minute: float) -> float:
        t = minute % 1440
        for p in self.speeds:
            if p["start"] <= t < p["end"]:
                return max(5.0, p["speed"])
        return 35.4
    def energy(self, km: float, speed: float, v: Vehicle,
load_ratio: float) -> Tuple[float, float, float]:
        speed = max(5.0, min(90.0, speed))
        load_ratio = max(0.0, min(1.0, load_ratio))
        load_factor = 1 + (v.load_multiplier - 1) *
load_ratio
        if v.energy == "fuel":
            per100 = (0.0025 * speed ** 2 - 0.2554
* speed + 31.75) * load_factor
            amount = per100 * km / 100
            cost = amount * self.cfg.fuel_price
            co2 = amount * self.cfg.fuel_co2
        else:
            per100 = (0.0014 * speed ** 2 - 0.12 *
speed + 36.19) * load_factor
            amount = per100 * km / 100
            cost = amount * self.cfg.elec_price
            co2 = amount * self.cfg.elec_co2

```



```

        return cost, co2, co2 * self.cfg.carbon_price
    def eval_departure(self, idx: int, v: Vehicle, stops:
List[Stop], depart: float) -> Route:
        r = Route(idx=idx, vehicle=v,
stops=copy.deepcopy(stops), depart=depart,
fixed=v.fixed_cost)
        t, cur, remain_w = depart, 0, sum(s.weight
for s in stops)
        for s in r.stops:
            km = self.d(cur, s.node_id)
            sp = self.speed(t)
            r.distance += km
            t += km / sp * 60
            e, co2, cc = self.energy(km, sp, v,
remain_w / v.weight_cap)
            r.energy_cost += e; r.emission += co2;
r.carbon_cost += cc
            if t < s.tw_start:
                r.wait_cost += (s.tw_start - t) / 60 *
self.cfg.wait_yuan_h
            t = s.tw_start
            elif t > s.tw_end:
                r.late_cost += (t - s.tw_end) / 60 *
self.cfg.late_yuan_h
            t += self.cfg.service_time
            remain_w -= s.weight
            cur = s.node_id
            km = self.d(cur, 0)
            sp = self.speed(t)
            r.distance += km
            t += km / sp * 60
            e, co2, cc = self.energy(km, sp, v, 0.0)
            r.energy_cost += e; r.emission += co2;
r.carbon_cost += cc
            r.back = t
            r.total = r.fixed + r.wait_cost + r.late_cost +
r.energy_cost + r.carbon_cost
            r.simple_total = r.fixed + r.energy_cost
            return r
    def best_departure(self, idx: int, v: Vehicle, stops:
List[Stop]) -> Route:
        candidates = {self.cfg.start_time}
        for s in stops:
            for off in [-180, -120, -60, -30, 0, 30]:

```

```

candidates.add(max(self.cfg.start_time, s.tw_start +
off))
            for off in [-120, -60, -30, 0]:
                candidates.add(max(self.cfg.start_time, s.tw_end +
off))
                upper = min(1320, max(s.tw_end for s in
stops))
                for m in range(int(self.cfg.start_time),
int(upper) + 1, self.cfg.depart_step):
                    candidates.add(float(m))
                    return min((self.eval_departure(idx, v, stops,
m) for m in candidates), key=lambda x: x.total)
            def nearest_order(self, stops: List[Stop]) ->
List[Stop]:
                left, cur, ans = stops[:], 0, []
                while left:
                    nxt = min(left, key=lambda s: self.d(cur,
s.node_id))
                    ans.append(nxt); left.remove(nxt); cur =
nxt.node_id
                return ans
            def best_order(self, idx: int, v: Vehicle, stops:
List[Stop]) -> Route:
                if len(stops) <= 1:
                    return self.best_departure(idx, v, stops)
                orders = [
                    stops[:],
                    sorted(stops, key=lambda s: s.tw_start),
                    sorted(stops, key=lambda s:
math.atan2(s.y - 20, s.x - 20)),
                    self.nearest_order(stops),
                ]
                best = min((self.best_departure(idx, v, o) for
o in orders), key=lambda x: x.total)
                if not self.cfg.use_2opt or len(best.stops) <
4:
                    return best
                order, improved = best.stops, True
                while improved:
                    improved = False
                    for i in range(len(order) - 2):
                        for j in range(i + 2, len(order) + 1):

```

```

        cand_order = order[:i] +
list(reversed(order[i:j])) + order[j:]
        cand = self.best_departure(idx,
v, cand_order)
        if cand.total + 1e-6 <
best.total:
            best, order, improved =
cand, cand.stops, True
            break
        if improved:
            break
    return best
# =====
# 五、启发式优化求解
# =====
class Solver:
    def __init__(self, customers: List[Customer],
vehicles: List[Vehicle], cost: CostModel, cfg: Config):
        self.customers = copy.deepcopy(customers)
        self.vehicles, self.cost, self.cfg = vehicles,
cost, cfg
        self.routes: List[Route] = []
    def left(self) -> List[Customer]:
        return [c for c in self.customers if
c.remain_w > 1e-6 and c.remain_v > 1e-6]
    def allowed(self, v: Vehicle, c: Customer) -> bool:
        if self.cfg.policy.lower() != "q2":
            return True
        if v.energy != "fuel" or not c.green:
            return True
        return not (c.tw_start < self.cfg.green_end
and c.tw_end > self.cfg.green_start)
    def vehicle_pool(self) -> List[Vehicle]:
        priority = {"EV-Heavy": 1, "Fuel-Heavy": 2,
"EV-Light": 3, "Fuel-Medium": 4, "Fuel-Light": 5}
        pool = []
        for v in sorted(self.vehicles, key=lambda x:
priority.get(x.name, 99)):
            pool += [v] * v.count
        return pool
    @staticmethod
    def take(c: Customer, rw: float, rv: float) ->
Optional[Stop]:
        if rw <= 1e-9 or rv <= 1e-9 or c.remain_w

```

```

<= 1e-9 or c.remain_v <= 1e-9:
            return None
        frac = min(1.0, rw / c.remain_w, rv /
c.remain_v)
        w, v = c.remain_w * frac, c.remain_v * frac
        c.remain_w -= w; c.remain_v -= v
        if c.remain_w < 1e-6: c.remain_w = 0.0
        if c.remain_v < 1e-6: c.remain_v = 0.0
        return Stop(c.node_id, c.x, c.y, w, v,
c.tw_start, c.tw_end, c.green)
    def build_route(self, idx: int, v: Vehicle) ->
Optional[Route]:
        feasible = [c for c in self.left() if
self.allowed(v, c)]
        if not feasible:
            return None
        seed = max(feasible, key=lambda c:
self.cost.d(0, c.node_id))
        rw, rv, stops, visited = v.weight_cap,
v.volume_cap, [], set()
        first = self.take(seed, rw, rv)
        if first is None:
            return None
        stops.append(first); visited.add(first.node_id)
        rw -= first.weight; rv -= first.volume
        while rw > 50 and rv > 0.02:
            candidates = [c for c in self.left() if
c.node_id not in visited and self.allowed(v, c)]
            if not candidates:
                break
            last = stops[-1]
            nxt = min(candidates, key=lambda c:
self.cost.d(last.node_id, c.node_id))
            stop = self.take(nxt, rw, rv)
            if stop is None:
                break
            stops.append(stop);
visited.add(stop.node_id)
            rw -= stop.weight; rv -= stop.volume
            if rw / v.weight_cap < 0.015 or rv /
v.volume_cap < 0.015:
                break
            return self.cost.best_order(idx, v, stops)
    def solve(self) -> Tuple[List[Route],

```

```

List[Customer]]:
    idx = 1
    for v in self.vehicle_pool():
        if not self.left():
            break
        route = self.build_route(idx, v)
        if route:
            self.routes.append(route)
            idx += 1
    return self.routes, self.left()
# =====
# 六、结果输出
# =====
class Reporter:
    def __init__(self, routes: List[Route], unserved:
List[Customer], original: List[Customer], nodes:
Dict[int, Tuple[float, float]], out: str | Path, runtime:
float):
        self.routes, self.unserved, self.original =
routes, unserved, original
        self.nodes, self.out, self.runtime = nodes,
mkdir(out), runtime
        def summary(self) -> Dict[str, float]:
            return {
                "完整模型总成本_元": sum(r.total for r
in self.routes),
                "简化对比成本_启动加能源_元":
sum(r.simple_total for r in self.routes),
                "车辆启动成本_元": sum(r.fixed for r
in self.routes),
                "等待惩罚成本_元": sum(r.wait_cost
for r in self.routes),
                "迟到惩罚成本_元": sum(r.late_cost
for r in self.routes),
                "能源消耗成本_元": sum(r.energy_cost
for r in self.routes),
                "碳排放成本_元": sum(r.carbon_cost
for r in self.routes),
                "碳排放量_kg": sum(r.emission for r in
self.routes),
                "总行驶距离_km": sum(r.distance for r
in self.routes),
                "使用车辆数": len(self.routes),
                "原始客户数": len(self.original),

```

```

                "未服务客户数": len(self.unserved),
                "已服务重量_kg": sum(r.weight for r in
self.routes),
                "总需求重量_kg": sum(c.weight for c
in self.original),
                "已服务体积_m3": sum(r.volume for r
in self.routes),
                "总需求体积_m3": sum(c.volume for c
in self.original),
                "运行时间_秒": self.runtime,
            }
        def print_report(self) -> None:
            s = self.summary()
            print("\n" + "=" * 62)
            print("城市绿色物流配送调度结果(简洁优
化版 v7) ")
            print("=" * 62)
            for k in ["完整模型总成本_元", "简化对比
成本_启动加能源_元", "使用车辆数", "总行驶距离
_km", "碳排放量_kg", "未服务客户数"]:
                print(f'{k:<22}: {s[k]:.2f}')
            print("\n 成本构成: ")
            for k in ["车辆启动成本_元", "等待惩罚成
本_元", "迟到惩罚成本_元", "能源消耗成本_元", "
碳排放成本_元"]:
                print(f'    {k:<18} {s[k]:.2f}')
            print("\n 车型使用情况: ")
            for name, count in
self.vehicle_usage().items():
                print(f'    {name:<10} {count} 辆')
            print("\n 前 10 条路径: ")
            for r in self.routes[:10]:
                print(f'    路径 {r.idx:03d} |
{r.vehicle.cn_name} | 成本 {r.total:.2f} | "
f"载重 {r.weight:.1f}kg | {' ->
'.join(map(str, r.path[:12]))} {' ...' if len(r.path) > 12 else
"}")
            def vehicle_usage(self) -> Dict[str, int]:
                ans: Dict[str, int] = {}
                for r in self.routes:
                    ans[r.vehicle.cn_name] =
ans.get(r.vehicle.cn_name, 0) + 1
                return dict(sorted(ans.items()))
            def save_excel(self) -> Path:

```

```

wb = Workbook()
ws = wb.active; ws.title = "结果汇总"
ws.append(["指标", "数值"])
for k, v in self.summary().items():
ws.append([k, v])
ws = wb.create_sheet("成本构成")
ws.append(["成本项目", "金额_元"])
for k in ["车辆启动成本_元", "等待惩罚成本_元", "迟到惩罚成本_元", "能源消耗成本_元", "碳排放成本_元"]:
ws.append([k.replace("_元", ""),
self.summary()[k]])
ws = wb.create_sheet("车型使用")
ws.append(["车型", "使用数量"])
for k, v in self.vehicle_usage().items():
ws.append([k, v])
ws = wb.create_sheet("路径明细")
ws.append(["路径编号", "车型", "客户路径", "出发时间", "返回时间", "载重 kg", "体积 m3", "距离 km", "总成本元", "能源成本元", "碳成本元", "等待成本元", "迟到成本元"])
for r in self.routes:
ws.append([r.idx, r.vehicle.cn_name, "
-> ".join(map(str, r.path)), minute_to_time(r.depart),
minute_to_time(r.back), r.weight, r.volume, r.distance,
r.total, r.energy_cost, r.carbon_cost, r.wait_cost,
r.late_cost])
ws = wb.create_sheet("未服务客户")
ws.append(["客户编号", "剩余重量 kg", "剩余体积 m3"])
for c in self.unserved: ws.append([c.node_id,
c.remain_w, c.remain_v])
path = self.out / "中文调度结果_简洁优化版 v7.xlsx"
wb.save(path)
return path
def draw_cost(self) -> Path:
setup_chinese_font()
items = ["车辆启动", "等待惩罚", "迟到惩罚", "能源消耗", "碳排放"]
vals = [sum(getattr(r, a) for r in self.routes)
for a in ["fixed", "wait_cost", "late_cost", "energy_cost", "carbon_cost"]]
plt.figure(figsize=(8, 5)); plt.bar(items, vals)

```

```

plt.title("成本构成图"); plt.ylabel("金额（元）"); plt.xticks(rotation=15); plt.tight_layout()
p = self.out / "成本构成图_v7.png";
plt.savefig(p, dpi=200); plt.close(); return p
def draw_vehicle(self) -> Path:
setup_chinese_font()
usage = self.vehicle_usage()
plt.figure(figsize=(8, 5));
plt.bar(list(usage.keys()), list(usage.values()))
plt.title("车型使用数量图"); plt.ylabel("车辆数"); plt.xticks(rotation=15); plt.tight_layout()
p = self.out / "车型使用图_v7.png";
plt.savefig(p, dpi=200); plt.close(); return p
def draw_map(self, max_routes: int = 80) -> Path:
setup_chinese_font()
plt.figure(figsize=(8, 7))
xs = [xy[0] for n, xy in self.nodes.items() if n != 0]
ys = [xy[1] for n, xy in self.nodes.items() if n != 0]
plt.scatter(xs, ys, s=12, label="客户点")
if 0 in self.nodes:
plt.scatter([self.nodes[0][0]], [self.nodes[0][1]], s=80, marker="*", label="配送中心")
for r in self.routes[:max_routes]:
pts = [self.nodes[n] for n in r.path if n in self.nodes]
plt.plot([p[0] for p in pts], [p[1] for p in pts], linewidth=0.5, alpha=0.35)
plt.title(f'配送路径空间分布图（展示前{min(max_routes, len(self.routes))} 条）')
plt.xlabel("X 坐标 km"); plt.ylabel("Y 坐标 km"); plt.legend(); plt.tight_layout()
p = self.out / "配送路径空间分布图_v7.png"; plt.savefig(p, dpi=200); plt.close(); return p
def save_all(self) -> Dict[str, Path]:
files = {"excel": self.save_excel(), "cost": self.draw_cost(), "vehicle": self.draw_vehicle(), "map": self.draw_map()}
txt = self.out / "运行结果摘要_v7.txt"
with open(txt, "w", encoding="utf-8") as f:
for k, v in self.summary().items():
f.write(f'{k}: {v}\n')

```

```

        files["txt"] = txt
        zip_path = self.out / "简洁优化版 v7_结果
打包.zip"
        with zipfile.ZipFile(zip_path, "w",
zipfile.ZIP_DEFLATED) as z:
            for p in files.values(): z.write(p,
arcname=p.name)
            files["zip"] = zip_path
        return files
# =====
# 七、主程序
# =====
def default_excel() -> Optional[Path]:
    names = ["A 题清洗过后的数据.xlsx", "A 题_第
一问_清洗后数据(1).xlsx"]
    for folder in [Path.cwd(),
Path(__file__).resolve().parent, Path("/mnt/data")]:
        for name in names:
            p = folder / name
            if p.exists(): return p
    return None
def parse_args():
    p = argparse.ArgumentParser(description="城市
绿色物流配送调度：简洁优化版 v7")
    p.add_argument("--input", default=None, help="
清洗后 Excel 文件路径")
    p.add_argument("--out", default="简洁优化版 v7
结果", help="输出文件夹")
    p.add_argument("--policy", default="q1",
choices=["q1", "q2"], help="q1=无政策限制, q2=绿色
区限行")
    p.add_argument("--no-2opt", action="store_true",
help="关闭 2-opt, 加快运行但结果可能变差")
    return p.parse_args()
def main() -> None:
    args = parse_args()
    excel = Path(args.input) if args.input else
default_excel()
    if not excel:
        raise FileNotFoundError("没有找到 Excel。
请使用 --input 指定文件路径。")
    t0 = time.perf_counter()
    loader = DataLoader(excel).load()
    loader.config.policy = args.policy

```

```

        loader.config.use_2opt = not args.no_2opt
        print(f"读取文件: {excel}")
        print(f"客户数: {len(loader.customers)}, 车型数:
{len(loader.vehicles)}, 政策: {args.policy}")
        cost = CostModel(loader.dist, loader.speeds,
loader.config)
        routes, unserved = Solver(loader.customers,
loader.vehicles, cost, loader.config).solve()
        runtime = time.perf_counter() - t0
        reporter = Reporter(routes, unserved,
loader.customers, loader.nodes, args.out, runtime)
        reporter.print_report()
        files = reporter.save_all()
        print("\n 输出文件: ")
        for k, p in files.items():
            print(f"    {k}: {p}")
        print("\n 程序运行完成。")
if __name__ == "__main__":
    main()

```

第二题求解代码：

```
# -*- coding: utf-8 -*-
```

```
"""
```

第二问：绿色配送区限行政策调度求解代码 v9_检查修正版

特点：

1. 代码比完整版更短，运行后直接打印中文结果；
2. 自动读取清洗后的 Excel；
3. 使用启发式优化：远端客户优先 + 最近邻填充 + 简单 2-opt + 出发时间搜索；
4. 自动生成 Excel 和可视化图片。

运行：

```
python 第二问_绿色配送区限行求解代码_v9_检查修正版.py --input "e:/数学建模/新 26A 题/A 题清洗过后的数据.xlsx"
```

```
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import copy
```

```
import math
```

```
import os
```

```
import time
```

```
import zipfile
```

```
from dataclasses import dataclass, field
```

```
from pathlib import Path
```

```
from typing import Dict, List, Optional, Tuple
```

```
try:
```

```
    from openpyxl import Workbook, load_workbook
```

```
except ImportError as exc:
```

```
    raise ImportError("缺少 openpyxl，请先运行：
```

```
pip install openpyxl") from exc
```

```
try:
```

```
    import matplotlib.pyplot as plt
```

```
except ImportError as exc:
```

```
    raise ImportError("缺少 matplotlib，请先运行：
```

```
pip install matplotlib") from exc
```

```
# =====
```

```
# 一、基础工具
```

```
# =====
```

```
def norm_name(s: object) -> str:
```

```
    """用于宽松匹配 Excel 工作表名。"""
```

```
    return str(s).lower().replace(" ", "").replace("_",
```

```
""").replace("-", "")
```

```
def minute_to_time(x: float | int | None) -> str:
```

```
    if x is None:
```

```
        return ""
```

```
    x = int(round(float(x))) % 1440
```

```
    return f"{x // 60:02d}:{x % 60:02d}"
```

```
def mkdir(path: str | Path) -> Path:
```

```
    p = Path(path)
```

```
    p.mkdir(parents=True, exist_ok=True)
```

```
    return p
```

```
def setup_chinese_font() -> None:
```

```
    """尽量让 matplotlib 支持中文。"""
```

```
    plt.rcParams["font.sans-serif"] = [
```

```
        "Microsoft YaHei", "SimHei", "Arial
```

```
Unicode MS", "DejaVu Sans"
```

```
]
```

```
    plt.rcParams["axes.unicode_minus"] = False
```

```
# =====
```

```
# 二、数据结构
```

```
# =====
```

```
@dataclass
```

```
class Vehicle:
```

```
    name: str
```

```
    energy: str
```

```
    weight_cap: float
```

```
    volume_cap: float
```

```
    count: int
```

```
    fixed_cost: float
```

```
    load_multiplier: float
```

```
    @property
```

```
    def cn_name(self) -> str:
```

```
        return {
```

```
            "Fuel-Heavy": "燃油重型车",
```

```
            "Fuel-Medium": "燃油中型车",
```

```
            "Fuel-Light": "燃油轻型车",
```

```
            "EV-Heavy": "新能源重型车",
```

```
            "EV-Light": "新能源轻型车",
```

```
        }.get(self.name, self.name)
```

```
@dataclass
```

```
class Customer:
```

```
    node_id: int
```

```
    x: float
```

```
    y: float
```

```
    weight: float
```

```

volume: float
tw_start: float
tw_end: float
green: bool
remain_w: float = field(init=False)
remain_v: float = field(init=False)
def __post_init__(self) -> None:
    self.remain_w = float(self.weight)
    self.remain_v = float(self.volume)
@dataclass
class Stop:
    node_id: int
    x: float
    y: float
    weight: float
    volume: float
    tw_start: float
    tw_end: float
    green: bool
@dataclass
class Route:
    idx: int
    vehicle: Vehicle
    stops: List[Stop]
    depart: float = 480.0
    back: float = 0.0
    distance: float = 0.0
    fixed: float = 0.0
    wait_cost: float = 0.0
    late_cost: float = 0.0
    energy_cost: float = 0.0
    carbon_cost: float = 0.0
    emission: float = 0.0
    total: float = 0.0
    simple_total: float = 0.0
    @property
    def path(self) -> List[int]:
        return [0] + [s.node_id for s in self.stops] +
[0]
    @property
    def weight(self) -> float:
        return sum(s.weight for s in self.stops)
    @property
    def volume(self) -> float:

```

```

        return sum(s.volume for s in self.stops)
@dataclass
class Config:
    policy: str = "q1"
    start_time: float = 480.0
    service_time: float = 20.0
    wait_yuan_h: float = 20.0
    late_yuan_h: float = 50.0
    fuel_price: float = 7.61
    elec_price: float = 1.64
    carbon_price: float = 0.65
    fuel_co2: float = 2.547
    elec_co2: float = 0.501
    depart_step: int = 30
    use_2opt: bool = True
    green_start: float = 480.0
    green_end: float = 960.0
# =====
# 三、Excel 读取
# =====
class DataLoader:
    def __init__(self, file_path: str | Path):
        self.file_path = Path(file_path)
        self.wb = None
        self.sheets: Dict[str, str] = {}
        self.customers: List[Customer] = []
        self.vehicles: List[Vehicle] = []
        self.speeds: List[Dict[str, float]] = []
        self.dist: Dict[int, Dict[int, float]] = {}
        self.nodes: Dict[int, Tuple[float, float]] = {}
        self.config = Config()
    def find_sheet(self, keyword: str) -> str:
        key = norm_name(keyword)
        for s in self.wb.sheetnames:
            if key in norm_name(s):
                return s
        raise ValueError(f"找不到包含 {keyword}
的工作表，当前工作表: {self.wb.sheetnames}")
    @staticmethod
    def rows_as_dicts(ws, header_row: int) ->
List[dict]:
        rows = list(ws.iter_rows(values_only=True))
        headers = [str(x) if x is not None else "" for x
in rows[header_row - 1]]

```

```

out = []
for row in rows[header_row:]:
    if not any(v is not None for v in row):
        continue
    out.append({headers[i]: row[i] if i <
len(row) else None for i in range(len(headers))})
return out
def load(self) -> "DataLoader":
    if not self.file_path.exists():
        raise FileNotFoundError(f"文件不存在:
{self.file_path}")
    self.wb = load_workbook(self.file_path,
read_only=True, data_only=True)
    self.sheets = {
        "customers": self.find_sheet("客户需求
汇总"),
        "nodes": self.find_sheet("全部节点含
仓库"),
        "dist": self.find_sheet("距离矩阵"),
        "params": self.find_sheet("车辆和参数
"),
    }
    self.load_customers()
    self.load_nodes()
    self.load_distance()
    self.load_params()
    return self
def load_customers(self) -> None:
    ws = self.wb[self.sheets["customers"]]
    for r in self.rows_as_dicts(ws, 1):
        if not bool(r.get("active_demand")):
            continue
        w = float(r.get("demand_weight_kg") or
0)
        v = float(r.get("demand_volume_m3")
or 0)
        if w <= 0 or v <= 0:
            continue
        self.customers.append(Customer(
            node_id=int(r["node_id"]),
            x=float(r["x_km"]),
            y=float(r["y_km"]),
            weight=w,
            volume=v,

```

```

tw_start=float(r.get("tw_start_min")
or 480),
        tw_end=float(r.get("tw_end_min")
or 1320),
        # 第二问补充说明要求: 绿色配
送区是以市中心(0,0)为圆心、半径 10km 的圆。
        # 因此这里直接根据坐标重新计
算, 而不是完全依赖 Excel 里的标记列。
        green=(math.sqrt(float(r["x_km"])
** 2 + float(r["y_km"]) ** 2) <= 10.0),
    ))
def load_nodes(self) -> None:
    ws = self.wb[self.sheets["nodes"]]
    for r in self.rows_as_dicts(ws, 1):
        if r.get("node_id") is None:
            continue
        self.nodes[int(r["node_id"])] =
(float(r["x_km"]), float(r["y_km"]))
    def load_distance(self) -> None:
        ws = self.wb[self.sheets["dist"]]
        rows = list(ws.iter_rows(values_only=True))
        heads = [int(x) for x in rows[0][1:] if x is not
None]
        for row in rows[1:]:
            if row[0] is None:
                continue
            i = int(row[0])
            self.dist[i] = {}
            for j, val in zip(heads, row[1:]):
                self.dist[i][j] = float(val or 0)
    def load_params(self) -> None:
        ws = self.wb[self.sheets["params"]]
        rows = list(ws.iter_rows(values_only=True))
        for idx, row in enumerate(rows):
            if row and row[0] == "vehicle_type":
                headers = list(row)
                for rr in rows[idx + 1:]:
                    if not rr or rr[0] is None:
                        break
                    d = dict(zip(headers, rr))
                    self.vehicles.append(Vehicle(
name=str(d["vehicle_type"]),

```



```

energy=str(d["energy_type"]),

weight_cap=float(d["capacity_weight_kg"]),

volume_cap=float(d["capacity_volume_m3"]),

count=int(d["available_quantity"]),

fixed_cost=float(d["fixed_start_cost_yuan_per_vehicle
"]),

load_multiplier=float(d["loaded_energy_multiplier"]),
    ))
    if row and row[0] == "period_type":
        headers = list(row)
        for rr in rows[idx + 1:]:
            if not rr or rr[0] is None:
                break
            d = dict(zip(headers, rr))
            self.speeds.append({
                "start":
float(d["start_min"]),
                "end":
float(d["end_min"]),
                "speed":
float(d["mean_km_h"]),
            })
        if row and row[0] == "parameter":
            headers = list(row)
            for rr in rows[idx + 1:]:
                if not rr or rr[0] is None:
                    break
                d = dict(zip(headers, rr))
                name = str(d["parameter"])
                try:
                    val = float(d["value"])
                except (TypeError,
ValueError):
                    continue
                if name == "service_time":
self.config.service_time = val
                elif name == "waiting_cost":
self.config.wait_yuan_h = val
                elif name ==

```

```

"late_penalty_cost": self.config.late_yuan_h = val
                elif name == "fuel_price":
self.config.fuel_price = val
                elif name ==
"electricity_price": self.config.elec_price = val
                elif name == "carbon_cost":
self.config.carbon_price = val
                elif name ==
"fuel_carbon_factor_eta": self.config.fuel_co2 = val
                elif name ==
"electric_carbon_factor_gamma": self.config.elec_co2
= val
                if not self.speeds:
                    self.speeds = [{ "start": 0, "end": 1440,
"speed": 35.4}]
# =====
# 四、成本计算
# =====
class CostModel:
    def __init__(self, dist: Dict[int, Dict[int, float]],
speeds: List[dict], cfg: Config):
        self.dist, self.speeds, self.cfg = dist, speeds,
cfg
    def d(self, i: int, j: int) -> float:
        return float(self.dist[int(i)][int(j)])
    def speed(self, minute: float) -> float:
        """返回当前分钟对应的平均车速。
        题目给出的速度是分时段正态分布。实际
求解中采用均值：
        顺畅 55.3 km/h, 一般 35.4 km/h, 拥堵 9.8
km/h。
        """
        t = minute % 1440
        for p in self.speeds:
            if p["start"] <= t < p["end"]:
                return max(5.0, p["speed"])
        # 若 Excel 只给出了 8:00-17:00 的速度，
则其他时间默认按一般时段处理。
        return 35.4
    def period_end(self, minute: float) -> float:
        """返回当前速度时段的结束分钟，用于跨
时段分段行驶计算。"""
        t = minute % 1440
        for p in self.speeds:

```

```

        if p["start"] <= t < p["end"]:
            return minute + (p["end"] - t)
        future_starts = [p["start"] for p in self.speeds
if p["start"] > t]
        if future_starts:
            return minute + (min(future_starts) - t)
        return minute + (1440 - t)
    def travel_segment(self, km: float, start_minute:
float, v: Vehicle, load_ratio: float) -> Tuple[float, float,
float, float]:
        """计算一段路程的分段行驶结果。
        返回:
            new_time: 到达时间, 单位分钟;
            energy_cost: 能源成本, 单位元;
            emission: 碳排放量, 单位 kg;
            carbon_cost: 碳排放成本, 单位元。
        这样处理后, 如果车辆从拥堵时段开到顺
        畅时段, 程序会自动按两个速度段分别计算,
        更符合数学建模文档中“跨越时间段时采
        用分段距离计算”的要求。
        """
        left_km = max(0.0, float(km))
        t = float(start_minute)
        energy_cost = 0.0
        emission = 0.0
        carbon_cost = 0.0
        guard = 0
        while left_km > 1e-9 and guard < 20:
            guard += 1
            sp = self.speed(t)
            end_t = self.period_end(t)
            available_min = max(1e-6, end_t - t)
            max_km_this_period = sp *
available_min / 60.0
            use_km = min(left_km,
max_km_this_period)
            e, co2, cc = self.energy(use_km, sp, v,
load_ratio)
            energy_cost += e
            emission += co2
            carbon_cost += cc
            t += use_km / sp * 60.0
            left_km -= use_km
        # 防御性兜底, 避免极端情况下没有速度

```

时段导致死循环。

```

        if left_km > 1e-9:
            sp = self.speed(t)
            e, co2, cc = self.energy(left_km, sp, v,
load_ratio)
            energy_cost += e
            emission += co2
            carbon_cost += cc
            t += left_km / sp * 60.0
        return t, energy_cost, emission, carbon_cost
    def energy(self, km: float, speed: float, v: Vehicle,
load_ratio: float) -> Tuple[float, float, float]:
        speed = max(5.0, min(90.0, speed))
        load_ratio = max(0.0, min(1.0, load_ratio))
        load_factor = 1 + (v.load_multiplier - 1) *
load_ratio
        if v.energy == "fuel":
            per100 = (0.0025 * speed ** 2 - 0.2554
* speed + 31.75) * load_factor
            amount = per100 * km / 100
            cost = amount * self.cfg.fuel_price
            co2 = amount * self.cfg.fuel_co2
        else:
            per100 = (0.0014 * speed ** 2 - 0.12 *
speed + 36.19) * load_factor
            amount = per100 * km / 100
            cost = amount * self.cfg.elec_price
            co2 = amount * self.cfg.elec_co2
        return cost, co2, co2 * self.cfg.carbon_price
    def eval_departure(self, idx: int, v: Vehicle, stops:
List[Stop], depart: float) -> Route:
        r = Route(idx=idx, vehicle=v,
stops=copy.deepcopy(stops), depart=depart,
fixed=v.fixed_cost)
        t, cur, remain_w = depart, 0, sum(s.weight
for s in stops)
        for s in r.stops:
            km = self.d(cur, s.node_id)
            r.distance += km
            t, e, co2, cc = self.travel_segment(km, t,
v, remain_w / v.weight_cap)
            r.energy_cost += e; r.emission += co2;
            r.carbon_cost += cc
            if t < s.tw_start:

```

```

        r.wait_cost += (s.tw_start - t) / 60 *
self.cfg.wait_yuan_h
        t = s.tw_start
        elif t > s.tw_end:
            r.late_cost += (t - s.tw_end) / 60 *
self.cfg.late_yuan_h
            t += self.cfg.service_time
            remain_w -= s.weight
            cur = s.node_id
            km = self.d(cur, 0)
            r.distance += km
            t, e, co2, cc = self.travel_segment(km, t, v,
0.0)
            r.energy_cost += e; r.emission += co2;
r.carbon_cost += cc
            r.back = t
            r.total = r.fixed + r.wait_cost + r.late_cost +
r.energy_cost + r.carbon_cost
            r.simple_total = r.fixed + r.energy_cost
            return r
        def best_departure(self, idx: int, v: Vehicle, stops:
List[Stop]) -> Route:
            candidates = {self.cfg.start_time}
            for s in stops:
                for off in [-180, -120, -60, -30, 0, 30]:
candidates.add(max(self.cfg.start_time, s.tw_start +
off))
                for off in [-120, -60, -30, 0]:
candidates.add(max(self.cfg.start_time, s.tw_end +
off))
            upper = min(1320, max(s.tw_end for s in
stops))
            for m in range(int(self.cfg.start_time),
int(upper) + 1, self.cfg.depart_step):
                candidates.add(float(m))
            return min((self.eval_departure(idx, v, stops,
m) for m in candidates), key=lambda x: x.total)
        def nearest_order(self, stops: List[Stop]) ->
List[Stop]:
            left, cur, ans = stops[:], 0, []
            while left:
                nxt = min(left, key=lambda s: self.d(cur,

```

```

s.node_id))
                ans.append(nxt); left.remove(nxt); cur =
nxt.node_id
            return ans
        def best_order(self, idx: int, v: Vehicle, stops:
List[Stop]) -> Route:
            if len(stops) <= 1:
                return self.best_departure(idx, v, stops)
            orders = [
                stops[:],
                sorted(stops, key=lambda s: s.tw_start),
                sorted(stops, key=lambda s:
math.atan2(s.y - 20, s.x - 20)),
                self.nearest_order(stops),
            ]
            best = min((self.best_departure(idx, v, o) for
o in orders), key=lambda x: x.total)
            if not self.cfg.use_2opt or len(best.stops) <
4:
                return best
            order, improved = best.stops, True
            while improved:
                improved = False
                for i in range(len(order) - 2):
                    for j in range(i + 2, len(order) + 1):
                        cand_order = order[:i] +
list(reversed(order[i:j])) + order[j:]
                        cand = self.best_departure(idx,
v, cand_order)
                        if cand.total + 1e-6 <
best.total:
                            best, order, improved =
cand, cand.stops, True
                            break
                        if improved:
                            break
                return best
# =====
# 五、启发式优化求解
# =====
class Solver:
    def __init__(self, customers: List[Customer],
vehicles: List[Vehicle], cost: CostModel, cfg: Config):
        self.customers = copy.deepcopy(customers)

```

```

        self.vehicles, self.cost, self.cfg = vehicles,
cost, cfg
        self.routes: List[Route] = []
        def left(self) -> List[Customer]:
            return [c for c in self.customers if
c.remain_w > 1e-6 and c.remain_v > 1e-6]
        def must_use_electric(self, c: Customer) -> bool:
            """第二问核心规则：绿色配送区内，且时
间窗与 8:00-16:00 禁行时段相交的客户，
视为必须由新能源车服务。
这是建模文档中“编程实现可直接强制绿
色区任务由新能源车承担”的简化线性化写法。
            """
            if self.cfg.policy.lower() != "q2":
                return False
            if not c.green:
                return False
            return c.tw_start < self.cfg.green_end and
c.tw_end > self.cfg.green_start
        def allowed(self, v: Vehicle, c: Customer) -> bool:
            """判断车辆 v 是否允许服务客户 c。
            第一问：所有车型均可服务所有客户。
            第二问：若客户属于绿色配送区，且服务
            时间窗与 8:00-16:00 限行时段相交，
            则燃油车不能服务该客户，只能由新能源
            车服务。
            """
            if self.cfg.policy.lower() != "q2":
                return True
            if self.must_use_electric(c) and v.energy ==
"fuel":
                return False
            return True
        def vehicle_pool(self) -> List[Vehicle]:
            priority = {"EV-Heavy": 1, "Fuel-Heavy": 2,
"EV-Light": 3, "Fuel-Medium": 4, "Fuel-Light": 5}
            pool = []
            for v in sorted(self.vehicles, key=lambda x:
priority.get(x.name, 99)):
                pool += [v] * v.count
            return pool
        @staticmethod
        def take(c: Customer, rw: float, rv: float) ->
Optional[Stop]:

```

```

            if rw <= 1e-9 or rv <= 1e-9 or c.remain_w
<= 1e-9 or c.remain_v <= 1e-9:
                return None
            frac = min(1.0, rw / c.remain_w, rv /
c.remain_v)
            w, v = c.remain_w * frac, c.remain_v * frac
            c.remain_w -= w; c.remain_v -= v
            if c.remain_w < 1e-6: c.remain_w = 0.0
            if c.remain_v < 1e-6: c.remain_v = 0.0
            return Stop(c.node_id, c.x, c.y, w, v,
c.tw_start, c.tw_end, c.green)
        def build_route(self, idx: int, v: Vehicle) ->
Optional[Route]:
            """构造单辆车路径。
            第二问优化重点：
            1. 新能源车优先装载“必须由新能源车服
务”的绿色区客户；
            2. 燃油车自动跳过绿色区限行客户；
            3. 每辆车按容量动态分担客户需求，允许
            大客户被多辆车分批服务。
            """
            left = self.left()
            # 若仍存在必须由新能源服务的绿色区客
            户，新能源车优先处理这些客户，
            # 避免新能源车被普通客户提前占用，导
            致绿色区客户未服务。
            must_ev_left = [c for c in left if
self.must_use_electric(c)]
            if self.cfg.policy.lower() == "q2" and
v.energy != "fuel" and must_ev_left:
                feasible = must_ev_left
            else:
                feasible = [c for c in left if
self.allowed(v, c)]
            if not feasible:
                return None
            # 远端客户优先作为种子，减少远距离客
            户单独派车的概率。
            seed = max(feasible, key=lambda c:
self.cost.d(0, c.node_id))
            rw, rv, stops, visited = v.weight_cap,
v.volume_cap, [], set()
            first = self.take(seed, rw, rv)
            if first is None:

```

```

        return None
    stops.append(first); visited.add(first.node_id)
    rw -= first.weight; rv -= first.volume
    while rw > 50 and rv > 0.02:
        left_now = self.left()
        must_ev_now = [c for c in left_now if
self.must_use_electric(c)]
        if self.cfg.policy.lower() == "q2" and
v.energy != "fuel" and must_ev_now:
            # 只要还有绿色限行客户没服务
完，新能源车继续优先装绿色限行客户。
            candidates = [c for c in
must_ev_now if c.node_id not in visited]
        else:
            candidates = [
                c for c in left_now
                if c.node_id not in visited and
self.allowed(v, c)
            ]
        if not candidates:
            break
        last = stops[-1]
        # 最近邻填充，使路径更短。
        nxt = min(candidates, key=lambda c:
self.cost.d(last.node_id, c.node_id))
        stop = self.take(nxt, rw, rv)
        if stop is None:
            break
        stops.append(stop);
visited.add(stop.node_id)
        rw -= stop.weight; rv -= stop.volume
        if rw / v.weight_cap < 0.015 or rv /
v.volume_cap < 0.015:
            break
        return self.cost.best_order(idx, v, stops)
    def solve(self) -> Tuple[List[Route],
List[Customer]]:
        idx = 1
        for v in self.vehicle_pool():
            if not self.left():
                break
            route = self.build_route(idx, v)
            if route:
                self.routes.append(route)

```

```

        idx += 1
        return self.routes, self.left()
# =====
# 六、结果输出
# =====
class Reporter:
    def __init__(self, routes: List[Route], unserved:
List[Customer], original: List[Customer], nodes:
Dict[int, Tuple[float, float]], out: str | Path, runtime:
float):
        self.routes, self.unserved, self.original =
routes, unserved, original
        self.nodes, self.out, self.runtime = nodes,
mkdir(out), runtime
    def summary(self) -> Dict[str, float]:
        return {
            "完整模型总成本_元": sum(r.total for r
in self.routes),
            "简化对比成本_启动加能源_元":
sum(r.simple_total for r in self.routes),
            "车辆启动成本_元": sum(r.fixed for r
in self.routes),
            "等待惩罚成本_元": sum(r.wait_cost
for r in self.routes),
            "迟到惩罚成本_元": sum(r.late_cost
for r in self.routes),
            "能源消耗成本_元": sum(r.energy_cost
for r in self.routes),
            "碳排放成本_元": sum(r.carbon_cost
for r in self.routes),
            "碳排放量_kg": sum(r.emission for r in
self.routes),
            "总行驶距离_km": sum(r.distance for r
in self.routes),
            "使用车辆数": len(self.routes),
            "原始客户数": len(self.original),
            "未服务客户数": len(self.unserved),
            "已服务重量_kg": sum(r.weight for r in
self.routes),
            "总需求重量_kg": sum(c.weight for c
in self.original),
            "已服务体积_m3": sum(r.volume for r
in self.routes),
            "总需求体积_m3": sum(c.volume for c

```

```

in self.original),
    "运行时间_秒": self.runtime,
}
def print_report(self) -> None:
    s = self.summary()
    print("\n" + "=" * 62)
    print("城市绿色物流配送调度结果(第二问绿色限行版 v9)")
    print("=" * 62)
    for k in ["完整模型总成本_元", "简化对比成本_启动加能源_元", "使用车辆数", "总行驶距离_km", "碳排放量_kg", "未服务客户数"]:
        print(f"{k:<22}: {s[k]:.2f}")
    print("\n 成本构成: ")
    for k in ["车辆启动成本_元", "等待惩罚成本_元", "迟到惩罚成本_元", "能源消耗成本_元", "碳排放成本_元"]:
        print(f"  {k:<18} {s[k]:.2f}")
    print("\n 车型使用情况: ")
    for name, count in self.vehicle_usage().items():
        print(f"  {name:<10} {count} 辆")
    print("\n 前 10 条路径: ")
    for r in self.routes[:10]:
        print(f"  路径 {r.idx:03d} | {r.vehicle.cn_name} | 成本 {r.total:,.2f} | "
              f"载重 {r.weight:,.1f}kg | {' -> ' + '.join(map(str, r.path[:12]))} {' ...' if len(r.path) > 12 else ''}")
    def vehicle_usage(self) -> Dict[str, int]:
        ans: Dict[str, int] = {}
        for r in self.routes:
            ans[r.vehicle.cn_name] = ans.get(r.vehicle.cn_name, 0) + 1
        return dict(sorted(ans.items()))
    def save_excel(self) -> Path:
        wb = Workbook()
        ws = wb.active; ws.title = "结果汇总"
        ws.append(["指标", "数值"])
        for k, v in self.summary().items():
            ws.append([k, v])
        ws = wb.create_sheet("成本构成")
        ws.append(["成本项目", "金额_元"])
        for k in ["车辆启动成本_元", "等待惩罚成本_元", "迟到惩罚成本_元", "能源消耗成本_元", "碳排放成本_元"]:

```

```

        ws.append([k.replace("_元", ""), ""],
self.summary()[k]])
        ws = wb.create_sheet("车型使用")
        ws.append(["车型", "使用数量"])
        for k, v in self.vehicle_usage().items():
            ws.append([k, v])
        ws = wb.create_sheet("路径明细")
        ws.append(["路径编号", "车型", "客户路径", "出发时间", "返回时间", "载重 kg", "体积 m3", "距离 km", "总成本元", "能源成本元", "碳成本元", "等待成本元", "迟到成本元"])
        for r in self.routes:
            ws.append([r.idx, r.vehicle.cn_name, " -> ".join(map(str, r.path)), minute_to_time(r.depart), minute_to_time(r.back), r.weight, r.volume, r.distance, r.total, r.energy_cost, r.carbon_cost, r.wait_cost, r.late_cost])
            ws = wb.create_sheet("未服务客户")
            ws.append(["客户编号", "剩余重量 kg", "剩余体积 m3"])
            for c in self.unserved: ws.append([c.node_id, c.remain_w, c.remain_v])
            path = self.out / "中文调度结果_第二问绿色限行版 v9.xlsx"
            wb.save(path)
            return path
    def draw_cost(self) -> Path:
        setup_chinese_font()
        items = ["车辆启动", "等待惩罚", "迟到惩罚", "能源消耗", "碳排放"]
        vals = [sum(getattr(r, a) for r in self.routes) for a in ["fixed", "wait_cost", "late_cost", "energy_cost", "carbon_cost"]]
        plt.figure(figsize=(8, 5)); plt.bar(items, vals)
        plt.title("成本构成图"); plt.ylabel("金额 (元)"); plt.xticks(rotation=15); plt.tight_layout()
        p = self.out / "成本构成图_第二问 v9.png";
        plt.savefig(p, dpi=200); plt.close(); return p
    def draw_vehicle(self) -> Path:
        setup_chinese_font()
        usage = self.vehicle_usage()
        plt.figure(figsize=(8, 5));

```

```

plt.bar(list(usage.keys()), list(usage.values()))
    plt.title("车型使用数量图"); plt.ylabel("车辆数"); plt.xticks(rotation=15); plt.tight_layout()
    p = self.out / "车型使用图_第二问 v9.png";
plt.savefig(p, dpi=200); plt.close(); return p
    def draw_map(self, max_routes: int = 80) -> Path:
        setup_chinese_font()
        plt.figure(figsize=(8, 7))
        xs = [xy[0] for n, xy in self.nodes.items() if
n != 0]
        ys = [xy[1] for n, xy in self.nodes.items() if
n != 0]
        plt.scatter(xs, ys, s=12, label="客户点")
        if 0 in self.nodes:
            plt.scatter([self.nodes[0][0]],
[self.nodes[0][1]], s=80, marker="*", label="配送中心")
            for r in self.routes[:max_routes]:
                pts = [self.nodes[n] for n in r.path if n in
self.nodes]
                plt.plot([p[0] for p in pts], [p[1] for p in
pts], linewidth=0.5, alpha=0.35)
                plt.title(f"配送路径空间分布图（展示前
{min(max_routes, len(self.routes))} 条）")
                plt.xlabel("X 坐标 km"); plt.ylabel("Y 坐标 km"); plt.legend(); plt.tight_layout()
                p = self.out / "配送路径空间分布图_第二
问 v9.png"; plt.savefig(p, dpi=200); plt.close(); return
p
            def save_all(self) -> Dict[str, Path]:
                files = {"excel": self.save_excel(), "cost":
self.draw_cost(), "vehicle": self.draw_vehicle(), "map":
self.draw_map()}
                txt = self.out / "运行结果摘要_第二问
v9.txt"
                with open(txt, "w", encoding="utf-8") as f:
                    for k, v in self.summary().items():
                        f.write(f"{k}: {v}\n")
                        files["txt"] = txt
                zip_path = self.out / "第二问绿色限行版
v9_结果打包.zip"
                with zipfile.ZipFile(zip_path, "w",
zipfile.ZIP_DEFLATED) as z:
                    for p in files.values(): z.write(p,

```

```

arcname=p.name)
                files["zip"] = zip_path
                return files
# =====
# 七、主程序
# =====
def default_excel() -> Optional[Path]:
    names = ["A 题清洗过后的数据.xlsx", "A 题_第
一问_清洗后数据(1).xlsx"]
    for folder in [Path.cwd(),
Path(__file__).resolve().parent, Path("/mnt/data")]:
        for name in names:
            p = folder / name
            if p.exists(): return p
        return None
def parse_args():
    p = argparse.ArgumentParser(description="第二
问: 绿色配送区限行政策调度求解代码 v9_检查修
正版")
    p.add_argument("--input", default=None, help="
清洗后 Excel 文件路径")
    p.add_argument("--out", default="第二问绿色限
行版 v9 结果", help="输出文件夹")
    p.add_argument("--no-2opt", action="store_true",
help="关闭 2-opt, 加快运行但结果可能略差")
    return p.parse_args()
def solve_one_policy(loader: DataLoader, policy: str,
out_dir: Path, use_2opt: bool):
    """按指定政策求解一次。
    policy='q1': 无绿色限行政策, 作为对比基准;
    policy='q2': 加入绿色配送区燃油车限行政策。
    """
    loader.config.policy = policy
    loader.config.use_2opt = use_2opt
    t0 = time.perf_counter()
    cost = CostModel(loader.dist, loader.speeds,
loader.config)
    routes, unserved = Solver(loader.customers,
loader.vehicles, cost, loader.config).solve()
    runtime = time.perf_counter() - t0
    reporter = Reporter(routes, unserved,
loader.customers, loader.nodes, out_dir, runtime)
    reporter.print_report()
    files = reporter.save_all()

```

```

        return reporter, files
def save_policy_comparison(q1: Reporter, q2:
Reporter, out_dir: Path) -> Path:
    """保存第一问与第二问政策影响对比表。"""
    out_dir = mkdir(out_dir)
    s1, s2 = q1.summary(), q2.summary()
    wb = Workbook()
    ws = wb.active
    ws.title = "政策对比分析"
    ws.append(["指标", "问题 1_无政策", "问题 2_
绿色限行", "变化量", "变化率%"])
    keys = [
        "完整模型总成本_元",
        "简化对比成本_启动加能源_元",
        "使用车辆数",
        "总行驶距离_km",
        "碳排放量_kg",
        "车辆启动成本_元",
        "等待惩罚成本_元",
        "迟到惩罚成本_元",
        "能源消耗成本_元",
        "碳排放成本_元",
        "未服务客户数",
    ]
    for k in keys:
        a, b = float(s1.get(k, 0)), float(s2.get(k, 0))
        rate = (b - a) / a * 100 if abs(a) > 1e-9 else ""
        ws.append([k, a, b, b - a, rate])
    ws2 = wb.create_sheet("车型结构对比")
    ws2.append(["车型", "问题 1 使用数量", "问题 2
使用数量", "变化量"])
    u1, u2 = q1.vehicle_usage(), q2.vehicle_usage()
    for name in sorted(set(u1) | set(u2)):
        ws2.append([name, u1.get(name, 0),
u2.get(name, 0), u2.get(name, 0) - u1.get(name, 0)])
    path = out_dir / "问题 1_问题 2_政策影响对
比.xlsx"
    wb.save(path)
    return path
def draw_policy_comparison(q1: Reporter, q2:
Reporter, out_dir: Path) -> Path:
    """绘制政策前后核心指标对比图。"""
    setup_chinese_font()
    s1, s2 = q1.summary(), q2.summary()

```

```

    labels = ["总成本", "启动成本", "能源成本", "碳
排放成本", "碳排放量"]
    q1_vals = [
        s1["完整模型总成本_元"],
        s1["车辆启动成本_元"],
        s1["能源消耗成本_元"],
        s1["碳排放成本_元"],
        s1["碳排放量_kg"],
    ]
    q2_vals = [
        s2["完整模型总成本_元"],
        s2["车辆启动成本_元"],
        s2["能源消耗成本_元"],
        s2["碳排放成本_元"],
        s2["碳排放量_kg"],
    ]
    x = list(range(len(labels)))
    width = 0.36
    plt.figure(figsize=(9, 5))
    plt.bar([i - width / 2 for i in x], q1_vals, width,
label="问题 1: 无政策")
    plt.bar([i + width / 2 for i in x], q2_vals, width,
label="问题 2: 绿色限行")
    plt.xticks(x, labels, rotation=15)
    plt.ylabel("金额/数量")
    plt.title("绿色配送政策前后核心指标对比")
    plt.legend()
    plt.tight_layout()
    path = out_dir / "问题 1_问题 2_政策影响对比
图.png"
    plt.savefig(path, dpi=200)
    plt.close()
    return path
def print_green_zone_info(loader: DataLoader) ->
None:
    active_green = [c for c in loader.customers if
c.green]
    must_ev = [
        c for c in loader.customers
        if c.green and c.tw_start <
loader.config.green_end and c.tw_end >
loader.config.green_start
    ]
    print("\n 绿色配送区识别: ")

```



```

    print(f' 半径 10km 内有需求客户数:
{len(active_green)}")
    print(f' 与 8:00-16:00 限行时段相交、需优先
新能源服务的客户数: {len(must_ev)}")
    print(" 绿色区客户编号: ", [c.node_id for c in
active_green])
def main() -> None:
    args = parse_args()
    excel = Path(args.input) if args.input else
default_excel()
    if not excel:
        raise FileNotFoundError("没有找到 Excel。
请使用 --input 指定文件路径。")
    out_root = mkdir(args.out)
    loader = DataLoader(excel).load()
    print("=" * 70)
    print("第二问: 绿色配送区限行政策下的车辆调
度求解")
    print("=" * 70)
    print(f'读取文件: {excel}')
    print(f'客户数: {len(loader.customers)}, 车型数:
{len(loader.vehicles)}")
    print_green_zone_info(loader)
    print("\n" + "=" * 70)
    print("先求解问题 1: 无政策限制, 作为基准")
    print("=" * 70)
    q1_reporter, q1_files = solve_one_policy(
        loader=loader,
        policy="q1",
        out_dir=out_root / "问题 1_无政策基准",
        use_2opt=not args.no_2opt,
    )
    print("\n" + "=" * 70)
    print("再求解问题 2: 加入绿色配送区燃油车限
行政策")
    print("=" * 70)
    q2_reporter, q2_files = solve_one_policy(
        loader=loader,
        policy="q2",
        out_dir=out_root / "问题 2_绿色限行方案",
        use_2opt=not args.no_2opt,
    )
    comparison_xlsx =
save_policy_comparison(q1_reporter, q2_reporter,

```

```

out_root)
    comparison_png =
draw_policy_comparison(q1_reporter, q2_reporter,
out_root)
    s1, s2 = q1_reporter.summary(),
q2_reporter.summary()
    dz = s2["完整模型总成本_元"] - s1["完整模型
总成本_元"]
    de = s2["碳排放量_kg"] - s1["碳排放量_kg"]
    cost_rate = dz / s1["完整模型总成本_元"] * 100
if s1["完整模型总成本_元"] else 0
    carbon_rate = de / s1["碳排放量_kg"] * 100 if
s1["碳排放量_kg"] else 0
    print("\n" + "=" * 70)
    print("第二问政策影响总结")
    print("=" * 70)
    print(f'问题 1 总成本: {s1['完整模型总成本_元
']:.,2f} 元")
    print(f'问题 2 总成本: {s2['完整模型总成本_元
']:.,2f} 元")
    print(f'成本变化: {dz:.,2f} 元, 变化率
{cost_rate:.,2f}%")
    print(f'问题 1 碳排放: {s1['碳排放量_kg']:.,2f}
kg")
    print(f'问题 2 碳排放: {s2['碳排放量_kg']:.,2f}
kg")
    print(f'碳排放变化: {de:.,2f} kg, 变化率
{carbon_rate:.,2f}%")
    print(f'问题 2 未服务客户数: {s2['未服务客户
数']:.,0f}")
    print("\n 输出文件: ")
    print(f' 问题 1 结果文件夹: {out_root / '问题
1_无政策基准'})
    print(f' 问题 2 结果文件夹: {out_root / '问题
2_绿色限行方案'})
    print(f' 政策对比 Excel: {comparison_xlsx}")
    print(f' 政策对比图片: {comparison_png}")
    print("\n 程序运行完成。")
if __name__ == "__main__":
    main()

```

第三题求解代码：

```
# -*- coding: utf-8 -*-  
"""
```

第三问：突发事件下的动态重调度求解代码 v1

用途：

1. 读取前两问使用的清洗后 Excel 数据；
2. 先构造绿色配送区政策下的基础调度方案；
3. 在指定触发时刻 t_e 发生新增订单/取消订单时，冻结已完成路径；
4. 基于车辆实时状态进行滚动重调度；
5. 采用“容量筛选 + 绿色区政策过滤 + 边际成本插入 + CVaR 风险评估”的方法选择最优插入方案；
6. 输出 Excel 结果表和可视化图片。

运行示例：

```
python 第三问_动态重调度求解代码_v1.py --input  
"e:/数学建模/新 26A 题/A 题清洗过后的数据.xlsx"  
可选参数：
```

--event-time 870 触发时刻，默认 870 分钟，即 14:30

--new-weight 500 新增订单重量 kg

--new-volume 1.5 新增订单体积 m3

--new-x 5 --new-y 5 新增订单坐标，默认在绿色配送区内

--cancel-node 0 取消订单客户编号，0 表示不执行取消事件

--samples 200 CVaR 蒙特卡洛模拟次数说明：

本代码为数学建模比赛可运行版本，采用启发式近似优化，不声明全局最优。

```
"""
```

```
from __future__ import annotations  
import argparse  
import math  
import os  
import random  
import shutil  
import zipfile  
from dataclasses import dataclass, field  
from pathlib import Path  
from typing import Dict, List, Optional, Tuple  
import numpy as np
```

```
import pandas as pd  
import matplotlib.pyplot as plt  
#  
=====
```

1. 基础工具函数

```
#  
=====
```

```
def setup_chinese_font() -> None:  
    """设置 Matplotlib 中文字体。"""  
    plt.rcParams["font.sans-serif"] = [  
        "Microsoft YaHei", "SimHei", "Arial  
Unicode MS", "DejaVu Sans"    ]
```

```
plt.rcParams["axes.unicode_minus"] = False  
def mkdir(path: str | Path) -> Path:
```

```
    p = Path(path)  
    p.mkdir(parents=True, exist_ok=True)  
    return p
```

```
def minute_to_time(x: float | int | None) -> str:
```

```
    """分钟数转 HH:MM 字符串。"""  
    if x is None or math.isinf(float(x)):  
        return ""
```

```
    x = int(round(float(x))) % 1440  
    return f"{x // 60:02d}:{x % 60:02d}"
```

```
def norm_name(s: object) -> str:
```

```
    """用于宽松匹配 Excel 工作表名。"""  
    return str(s).lower().replace(" ", "").replace("_",  
        "").replace("-", "")
```

```
def in_interval(t: float, start: float, end: float) -> bool:  
    return start <= t < end
```

```
#  
=====
```

2. 数据结构

```
#  
=====
```

```
@dataclass  
class Vehicle:  
    """车辆类型。"""  
    name: str  
    cn_name: str
```

```

energy: str # fuel/electric
weight_cap: float
volume_cap: float
count: int
fixed_cost: float = 400.0
@dataclass
class Customer:
    """客户需求。"""
    node_id: int
    x: float
    y: float
    weight: float
    volume: float
    tw_start: float
    tw_end: float
    green: bool
    remain_w: float = field(init=False)
    remain_v: float = field(init=False)
    def __post_init__(self) -> None:
        self.remain_w = float(self.weight)
        self.remain_v = float(self.volume)
@dataclass
class Stop:
    """路径中的一个配送任务。允许同一客户被拆
分到多辆车。"""
    node_id: int
    x: float
    y: float
    weight: float
    volume: float
    tw_start: float
    tw_end: float
    green: bool
    label: str = "原始订单"
@dataclass
class Route:
    """完整车辆路径。"""
    route_id: int
    vehicle: Vehicle
    stops: List[Stop]
    depart: float = 480.0
    fixed_cost: float = 400.0
    distance: float = 0.0
    wait_cost: float = 0.0

```

```

late_cost: float = 0.0
energy_cost: float = 0.0
carbon_cost: float = 0.0
emission: float = 0.0
total_cost: float = 0.0
back_time: float = 0.0
schedule: List[dict] = field(default_factory=list)
@property
def weight(self) -> float:
    return sum(s.weight for s in self.stops)
@property
def volume(self) -> float:
    return sum(s.volume for s in self.stops)
@property
def path_text(self) -> str:
    return "0 -> " + " -> ".join(str(s.node_id) for
s in self.stops) + " -> 0"
@dataclass
class VehicleState:
    """触发时刻 te 的车辆状态。"""
    route_id: int
    vehicle: Vehicle
    current_node: int
    current_x: float
    current_y: float
    available_time: float
    completed_stops: List[Stop]
    remaining_stops: List[Stop]
    remaining_weight: float
    remaining_volume: float
    spare_weight: float
    spare_volume: float
@dataclass
class Config:
    """模型参数。"""
    service_time: float = 20.0
    wait_yuan_h: float = 20.0
    late_yuan_h: float = 50.0
    fuel_price: float = 7.61
    elec_price: float = 1.64
    carbon_price: float = 0.65
    fuel_co2: float = 2.547
    elec_co2: float = 0.501
    green_start: float = 480.0 # 8:00

```

```

green_end: float = 960.0          # 16:00
risk_alpha: float = 0.95
risk_lambda: float = 0.15
route_change_penalty: float = 50.0
carbon_quota: float = 9000.0
carbon_reward_eta: float = 0.02
#
=====
=====
# 3. 数据读取
#
=====
=====
class DataLoader:
    """读取清洗后的 Excel 数据。"""
    def __init__(self, excel_path: str | Path):
        self.excel_path = Path(excel_path)
        self.customers: List[Customer] = []
        self.nodes: Dict[int, Tuple[float, float]] = {}
        self.dist: Dict[int, Dict[int, float]] = {}
        self.vehicles: List[Vehicle] = []
    def _find_sheet(self, xls: pd.ExcelFile, keyword:
str) -> str:
        key = norm_name(keyword)
        for s in xls.sheet_names:
            if key in norm_name(s):
                return s
        raise ValueError(f"找不到包含 '{keyword}'
的工作表，当前工作表：{xls.sheet_names}")
    def load(self) -> "DataLoader":
        if not self.excel_path.exists():
            raise FileNotFoundError(f"Excel 文件
不存在：{self.excel_path}")
        xls = pd.ExcelFile(self.excel_path)
        sheet_customer = self._find_sheet(xls, "客户
需求汇总")
        sheet_nodes = self._find_sheet(xls, "全部节
点含仓库")
        sheet_dist = self._find_sheet(xls, "距离矩阵
")
        cust_df = pd.read_excel(self.excel_path,
sheet_name=sheet_customer)
        node_df = pd.read_excel(self.excel_path,
sheet_name=sheet_nodes)

```

```

dist_df = pd.read_excel(self.excel_path,
sheet_name=sheet_dist)
# 节点坐标
for _, row in node_df.iterrows():
    node_id = int(row.get("node_id",
row.get("ID", 0)))
    x = float(row.get("x_km", row.get("X
(km)", 0)))
    y = float(row.get("y_km", row.get("Y
(km)", 0)))
    self.nodes[node_id] = (x, y)
# 客户需求
for _, row in cust_df.iterrows():
    node_type = str(row.get("node_type", "
客户"))
    active = bool(row.get("active_demand",
True))
    node_id = int(row.get("node_id",
row.get("客户编号", 0)))
    weight =
float(row.get("demand_weight_kg", row.get("总重量
_kg", 0)) or 0)
    volume =
float(row.get("demand_volume_m3", row.get("总体积
_m3", 0)) or 0)
    if node_id == 0 or "仓库" in node_type
or not active or weight <= 0:
        continue
    x = float(row.get("x_km",
self.nodes.get(node_id, (0, 0))[0]))
    y = float(row.get("y_km",
self.nodes.get(node_id, (0, 0))[1]))
    tw_start = float(row.get("tw_start_min",
480) or 480)
    tw_end = float(row.get("tw_end_min",
1020) or 1020)
    green = math.sqrt(x * x + y * y) <= 10.0
    self.customers.append(
        Customer(node_id, x, y, weight,
volume, tw_start, tw_end, green)
    )
    self.nodes[node_id] = (x, y)
# 距离矩阵，兼容第一列 from_to 的格式
if "from_to" in dist_df.columns:

```

```

        dist_df = dist_df.set_index("from_to")
    else:
        dist_df =
dist_df.set_index(dist_df.columns[0])
    for i in dist_df.index:
        try:
            from_id = int(i)
        except Exception:
            continue
        self.dist[from_id] = {}
        for col in dist_df.columns:
            try:
                to_id = int(col)
                value = float(dist_df.loc[i,
col])
                self.dist[from_id][to_id] =
value
            except Exception:
                continue
        self.vehicles = [
            Vehicle("Fuel-Heavy", "燃油重型车",
"fuel", 3000, 13.5, 60),
            Vehicle("Fuel-Medium", "燃油中型车",
"fuel", 1500, 10.8, 50),
            Vehicle("Fuel-Light", "燃油轻型车",
"fuel", 1250, 6.5, 50),
            Vehicle("EV-Heavy", "新能源重型车",
"electric", 3000, 15.0, 10),
            Vehicle("EV-Light", "新能源轻型车",
"electric", 1250, 8.5, 15),
        ]
        print(f'成功读取客户数:
{len(self.customers)}')
        print(f'绿色配送区客户数: {sum(1 for c in
self.customers if c.green)}')
        print(f'距离矩阵节点数: {len(self.dist)}')
        return self
    def distance(self, a: int, b: int) -> float:
        """优先读取距离矩阵; 若新订单不在矩阵
中, 则用欧氏距离估算。"""
        if a in self.dist and b in self.dist[a]:
            return float(self.dist[a][b])
        ax, ay = self.nodes.get(a, (0.0, 0.0))
        bx, by = self.nodes.get(b, (0.0, 0.0))

```

```

        return math.hypot(ax - bx, ay - by)
#
=====
=====
# 4. 成本与时间模型
#
=====
=====
class CostModel:
    """时变速度、能耗、碳排放与时间窗惩罚模型。
    """
    def __init__(self, loader: DataLoader, config:
Config):
        self.data = loader
        self.cfg = config
    def speed_profile(self, t: float, random_mode:
bool = False) -> Tuple[float, float]:
        """返回当前时刻速度及当前时段结束时间。random_mode=True 时返回随机速度。"""
        m = t % 1440
        periods = [
            (480, 540, 9.8, 4.7),          # 8:00-9:00
            拥堵
            (540, 600, 55.3, 0.1),         #
            9:00-10:00 顺畅
            (600, 690, 35.4, 5.2),         #
            10:00-11:30 一般
            (690, 780, 9.8, 4.7),          #
            11:30-13:00 拥堵
            (780, 900, 55.3, 0.1),         #
            13:00-15:00 顺畅
            (900, 1020, 35.4, 5.2),        #
            15:00-17:00 一般
        ]
        for start, end, mean, std in periods:
            if start <= m < end:
                speed = np.random.normal(mean,
std) if random_mode else mean
                return max(3.0, float(speed)), t +
(end - m)
        return 35.4, math.floor(t / 1440) * 1440 +
1440
    def travel_time(self, distance_km: float,
start_time: float, random_mode: bool = False) -> float:

```

```

        """跨时段分段计算行驶时间，单位：分钟。
        """
        remain = max(0.0, distance_km)
        t = float(start_time)
        guard = 0
        while remain > 1e-8 and guard < 100:
            guard += 1
            speed, period_end = self.speed_profile(t,
random_mode)
            available_min = max(1e-6, period_end -
t)

            can_run = speed * available_min / 60.0
            if remain <= can_run:
                t += remain / speed * 60.0
                remain = 0.0
            else:
                remain -= can_run
                t = period_end
        return t - start_time

    def energy_and_carbon(self, distance_km: float,
vehicle: Vehicle, load_ratio: float,
                        avg_speed: float) ->
Tuple[float, float, float]:
        """返回 energy_cost, carbon_cost, emission。
        """
        v = max(3.0, min(80.0, avg_speed))
        if vehicle.energy == "fuel":
            rate = 0.0025 * v ** 2 - 0.2554 * v +
31.75

            rate *= (1 + 0.4 * load_ratio)
            fuel_l = rate / 100.0 * distance_km
            energy_cost = fuel_l *

self.cfg.fuel_price
            emission = fuel_l * self.cfg.fuel_co2
        else:
            rate = 0.0014 * v ** 2 - 0.12 * v +
36.19

            rate *= (1 + 0.35 * load_ratio)
            elec_kwh = rate / 100.0 * distance_km
            energy_cost = elec_kwh *

self.cfg.elec_price
            emission = elec_kwh *

self.cfg.elec_co2
            carbon_cost = emission *

```

```

self.cfg.carbon_price
        return energy_cost, carbon_cost, emission

    def service_violates_green_policy(self, vehicle:
Vehicle, stop: Stop, arrival: float) -> bool:
        """检查燃油车是否在 8:00-16:00 服务绿
色区客户。

        注意：若车辆早到并等待，真正的服务开
始时间不是 arrival，
        而是 max(arrival, stop.tw_start)。这一点会
影响绿色区限行判断。
        """
        if vehicle.energy != "fuel" or not stop.green:
            return False
        service_start = max(arrival, stop.tw_start)
        service_end = service_start +

self.cfg.service_time
        return service_start < self.cfg.green_end and
service_end > self.cfg.green_start

    def evaluate_sequence(self, vehicle: Vehicle,
stops: List[Stop], depart: float,
                        start_node: int = 0,
include_fixed: bool = True,
                        random_mode: bool =
False,
                        return_to_depot: bool
= True) -> dict:
        """计算某车辆从 start_node 出发服务
stops 的成本。

        return_to_depot=False 用于第三问“冻结已
完成路径”：
        已完成部分只计算到当前节点，不强行返
回仓库。
        """
        current = start_node
        t = float(depart)
        remaining_weight = sum(s.weight for s in
stops)

        total_distance = 0.0
        wait_cost = 0.0
        late_cost = 0.0
        energy_cost = 0.0
        carbon_cost = 0.0
        emission = 0.0
        schedule = []

```

```

violation = 0
for stop in stops:
    dist = self.data.distance(current,
stop.node_id)
    travel = self.travel_time(dist, t,
random_mode=random_mode)
    avg_speed = dist / travel * 60.0 if
travel > 1e-6 else 35.4
    load_ratio = min(1.0, max(0.0,
remaining_weight / vehicle.weight_cap))
    e_cost, c_cost, co2 =
self.energy_and_carbon(dist, vehicle, load_ratio,
avg_speed)
    energy_cost += e_cost
    carbon_cost += c_cost
    emission += co2
    total_distance += dist
    arrival = t + travel
    if
self.service_violates_green_policy(vehicle, stop,
arrival):
        violation += 1
        wait = max(0.0, stop.tw_start - arrival)
        late = max(0.0, arrival - stop.tw_end)
        wait_cost += wait / 60.0 *
self.cfg.wait_yuan_h
        late_cost += late / 60.0 *
self.cfg.late_yuan_h
        service_start = max(arrival,
stop.tw_start)
        depart_stop = service_start +
self.cfg.service_time
        schedule.append({
            "node_id": stop.node_id,
            "到达时间": arrival,
            "开始服务": service_start,
            "离开时间": depart_stop,
            "等待分钟": wait,
            "迟到分钟": late,
            "是否绿色区": stop.green,
            "燃油车限行违规": violation > 0,
        })
        t = depart_stop
        remaining_weight -= stop.weight

```

```

current = stop.node_id
# 是否返回仓库。静态完整路径需要返回；
动态已完成部分不需要返回。
if return_to_depot:
    dist_back = self.data.distance(current,
0)
    travel_back = self.travel_time(dist_back,
t, random_mode=random_mode)
    avg_speed = dist_back / travel_back *
60.0 if travel_back > 1e-6 else 35.4
    e_cost, c_cost, co2 =
self.energy_and_carbon(dist_back, vehicle, 0.0,
avg_speed)
    energy_cost += e_cost
    carbon_cost += c_cost
    emission += co2
    total_distance += dist_back
    back_time = t + travel_back
else:
    back_time = t
    fixed = vehicle.fixed_cost if include_fixed
else 0.0
    total = fixed + wait_cost + late_cost +
energy_cost + carbon_cost
    return {
        "distance": total_distance,
        "wait_cost": wait_cost,
        "late_cost": late_cost,
        "energy_cost": energy_cost,
        "carbon_cost": carbon_cost,
        "emission": emission,
        "fixed_cost": fixed,
        "total_cost": total,
        "back_time": back_time,
        "schedule": schedule,
        "violation": violation,
    }
#
=====
# 5. 基础路径生成器
#
=====

```

```

class StaticHeuristicSolver:
    """构造第三问的基础调度方案。默认使用问题
    2 绿色限行政策。"""
    def __init__(self, loader: DataLoader, cost_model:
    CostModel, cfg: Config):
        self.data = loader
        self.cost = cost_model
        self.cfg = cfg
    def _take_stop(self, customer: Customer, w: float,
    v: float) -> Stop:
        return Stop(
            node_id=customer.node_id,
            x=customer.x,
            y=customer.y,
            weight=w,
            volume=v,
            tw_start=customer.tw_start,
            tw_end=customer.tw_end,
            green=customer.green,
        )
    def build_one_route(self, vehicle: Vehicle,
    customers: List[Customer],
                        depart: float,
    force_green_policy: bool = True) -> Optional[Route]:
        """远端客户优先 + 最近邻填充，构造一
        条路径。"""
        remaining_w = vehicle.weight_cap
        remaining_v = vehicle.volume_cap
        stops: List[Stop] = []
        current_node = 0
        while True:
            candidates = []
            for c in customers:
                if c.remain_w <= 1e-6 or
c.remain_v <= 1e-6:
                    continue
                if force_green_policy and
vehicle.energy == "fuel" and c.green and depart <
self.cfg.green_end:
                    # 第三问基准方案采用更高
                    效的编程约束：限行期内绿色区任务优先给新能源
                    车
                    continue
                if remaining_w <= 1e-6 or

```

```

remaining_v <= 1e-6:
                    continue
                # 按同一比例拆分客户需求，避
                免“重量拆了、体积没按比例拆”的口径错误。
                frac_w = remaining_w /
c.remain_w if c.remain_w > 0 else 0.0
                frac_v = remaining_v / c.remain_v
                if c.remain_v > 0 else 0.0
                take_frac = min(1.0, frac_w,
                frac_v)
                take_w = c.remain_w * take_frac
                take_v = c.remain_v * take_frac
                if take_w > 1e-6 and take_v >
1e-6:
                    d =
self.data.distance(current_node, c.node_id)
                    depot_d = self.data.distance(0,
c.node_id)
                    # 首个客户偏向远端，后续
                    偏向最近邻
                    score = depot_d if not stops
                else -d
                    candidates.append((score, d, c,
                take_w, take_v))
                if not candidates:
                    break
                if not stops:
                    candidates.sort(key=lambda x:
x[0], reverse=True)
                else:
                    candidates.sort(key=lambda x:
x[1])
                _, _, chosen, take_w, take_v =
candidates[0]
                stops.append(self._take_stop(chosen,
                take_w, take_v))
                chosen.remain_w -= take_w
                chosen.remain_v -= take_v
                remaining_w -= take_w
                remaining_v -= take_v
                current_node = chosen.node_id
                if remaining_w < 1e-6 or remaining_v <
1e-6:
                    break

```



```

        if not stops:
            return None
        route = Route(route_id=-1, vehicle=vehicle,
stops=stops, depart=depart)
        metrics =
self.cost.evaluate_sequence(vehicle, stops, depart,
include_fixed=True)
        self._fill_route_metrics(route, metrics)
        return route
    @staticmethod
    def _fill_route_metrics(route: Route, metrics: dict)
-> None:
        route.fixed_cost = metrics["fixed_cost"]
        route.distance = metrics["distance"]
        route.wait_cost = metrics["wait_cost"]
        route.late_cost = metrics["late_cost"]
        route.energy_cost = metrics["energy_cost"]
        route.carbon_cost = metrics["carbon_cost"]
        route.emission = metrics["emission"]
        route.total_cost = metrics["total_cost"]
        route.back_time = metrics["back_time"]
        route.schedule = metrics["schedule"]
    def solve(self) -> List[Route]:
        """生成基础静态路线。"""
        customers = [Customer(c.node_id, c.x, c.y,
c.weight, c.volume, c.tw_start, c.tw_end, c.green)
for c in self.data.customers]
        # 策略：新能源优先处理绿色区，随后燃油
重型车提高装载率
        order = ["EV-Heavy", "EV-Light",
"Fuel-Heavy", "Fuel-Medium", "Fuel-Light"]
        vehicles = []
        for name in order:
            vehicles.extend([v for v in
self.data.vehicles if v.name == name])
        routes: List[Route] = []
        for vehicle in vehicles:
            for _ in range(vehicle.count):
                # 燃油车若还有绿色区剩余，可
以在 16:00 后服务；否则 8:00 出发
                green_left = any(c.green and
c.remain_w > 1e-6 for c in customers)
                depart = 960.0 if vehicle.energy ==
"fuel" and green_left else 480.0

```

```

        route =
self.build_one_route(vehicle, customers, depart,
force_green_policy=True)
        if route is None:
            break
        route.route_id = len(routes) + 1
        routes.append(route)
        if not any(c.remain_w > 1e-4 for c
in customers):
            return routes
        return routes
#
=====
# 6. 动态重调度器
#
=====
class DynamicDispatcher:
    """第三问滚动重调度核心。"""
    def __init__(self, loader: DataLoader, cost_model:
CostModel, cfg: Config):
        self.data = loader
        self.cost = cost_model
        self.cfg = cfg
    def get_vehicle_states(self, routes: List[Route],
event_time: float) -> List[VehicleState]:
        """冻结 event_time 前已完成任务, 提取车
辆实时状态。"""
        states = []
        for r in routes:
            completed: List[Stop] = []
            remaining: List[Stop] = []
            current_node = 0
            current_x, current_y =
self.data.nodes.get(0, (0.0, 0.0))
            available_time = event_time
            for stop, sch in zip(r.stops, r.schedule):
                if sch["离开时间"] <= event_time:
                    completed.append(stop)
                    current_node = stop.node_id
                    current_x, current_y = stop.x,
stop.y
                    available_time =

```

```

max(event_time, sch["离开时间"])
        else:
            remaining.append(stop)
            remaining_w = sum(s.weight for s in
remaining)
            remaining_v = sum(s.volume for s in
remaining)
            states.append(
                VehicleState(
                    route_id=r.route_id,
                    vehicle=r.vehicle,
                    current_node=current_node,
                    current_x=current_x,
                    current_y=current_y,

available_time=available_time,
                    completed_stops=completed,
                    remaining_stops=remaining,

remaining_weight=remaining_w,

remaining_volume=remaining_v,

spare_weight=r.vehicle.weight_cap - remaining_w,

spare_volume=r.vehicle.volume_cap - remaining_v,
                )
            )
        return states
    def cvar_late_penalty(self, vehicle: Vehicle,
start_node: int, stops: List[Stop],
                    start_time: float,
samples: int) -> Tuple[float, float, List[float]]:
    """蒙特卡洛模拟时变速度，计算迟到罚金
CVaR。"""
    penalties = []
    for _ in range(samples):
        m = self.cost.evaluate_sequence(
            vehicle, stops, start_time,
start_node=start_node,
                    include_fixed=False,
random_mode=True
        )
        penalties.append(float(m["late_cost"]))

```

```

arr = np.array(penalties, dtype=float)
expected = float(arr.mean()) if len(arr) else
0.0
    if len(arr) == 0:
        return expected, 0.0, penalties
    threshold = np.quantile(arr,
self.cfg.risk_alpha)
    tail = arr[arr >= threshold]
    cvar = float(tail.mean()) if len(tail) else
expected
        return expected, cvar, penalties
    def _combine_dynamic_metrics(self, route: Route,
state: VehicleState,

new_remaining: List[Stop]) -> None:
    """合并动态重调度后的成本。
    第三问要求冻结 event_time 前已完成路
径，所以不能把“完成节点+新增节点”
重新当成一条从 8:00 出发的全新路线来
计算。这里分两段：
        1) 已完成部分：从仓库出发到当前节点，
不返回仓库；
        2) 剩余部分：从当前节点、当前时刻出发，
服务剩余任务后返回仓库。
    """
    completed_metrics =
self.cost.evaluate_sequence(
        route.vehicle, state.completed_stops,
route.depart,
        start_node=0, include_fixed=True,
return_to_depot=False
    )
    remaining_metrics =
self.cost.evaluate_sequence(
        route.vehicle, new_remaining,
state.available_time,
        start_node=state.current_node,
include_fixed=False, return_to_depot=True
    )
    combined = {}
    for key in [
        "distance", "wait_cost", "late_cost",
"energy_cost",
        "carbon_cost", "emission", "fixed_cost",

```

```

"total_cost", "violation"
    ]:
        combined[key] =
completed_metrics[key] + remaining_metrics[key]
        combined["back_time"] =
remaining_metrics["back_time"]
        combined["schedule"] =
completed_metrics["schedule"] +
remaining_metrics["schedule"]
        route.stops = state.completed_stops +
new_remaining

StaticHeuristicSolver._fill_route_metrics(route,
combined)
    def apply_cancel_event(self, routes: List[Route],
cancel_node: int, event_time: float) ->
Tuple[List[Route], dict]:
        """取消订单：只取消尚未服务的该客户任
务。"""
        if cancel_node <= 0:
            return routes, {"是否执行取消": "否"}
        removed_w = 0.0
        removed_v = 0.0
        affected_route = None
        for r in routes:
            new_stops = []
            for stop, sch in zip(r.stops, r.schedule):
                if stop.node_id == cancel_node
and sch["离开时间"] > event_time:
                    removed_w += stop.weight
                    removed_v += stop.volume
                    affected_route = r.route_id
                    continue
            new_stops.append(stop)
            r.stops = new_stops
            metrics =
self.cost.evaluate_sequence(r.vehicle, r.stops, r.depart,
include_fixed=True)

StaticHeuristicSolver._fill_route_metrics(r, metrics)
    return routes, {
        "是否执行取消": "是",
        "取消客户编号": cancel_node,
        "影响路线": affected_route if

```

```

affected_route else "未找到未完成任务",
        "取消重量 kg": removed_w,
        "取消体积 m3": removed_v,
    }
    def insert_new_order(self, routes: List[Route],
new_stop: Stop,
                                event_time: float,
samples: int) -> Tuple[List[Route], pd.DataFrame, dict,
List[float]]:
        """新增订单：在现有路径中寻找最低综合
边际成本插入位置。"""
        states = self.get_vehicle_states(routes,
event_time)
        evaluations = []
        best = None
        best_penalties: List[float] = []
        for st in states:
            if st.spare_weight + 1e-6 <
new_stop.weight or st.spare_volume + 1e-6 <
new_stop.volume:
                continue
            if new_stop.green and st.vehicle.energy
== "fuel" and in_interval(event_time,
self.cfg.green_start, self.cfg.green_end):
                continue
            old_metrics =
self.cost.evaluate_sequence(
                st.vehicle, st.remaining_stops,
st.available_time,
                start_node=st.current_node,
include_fixed=False
            )
            for pos in range(len(st.remaining_stops)
+ 1):
                candidate_stops =
st.remaining_stops[:pos] + [new_stop] +
st.remaining_stops[pos:]
                # 政策硬检查
                check =
self.cost.evaluate_sequence(
                st.vehicle, candidate_stops,
st.available_time,
                start_node=st.current_node,
include_fixed=False

```

```

        )
        if check["violation"] > 0:
            continue
        exp_late, cvar, penalties =
self.cvar_late_penalty(
            st.vehicle, st.current_node,
candidate_stops,
            st.available_time, samples
        )
        marginal = check["total_cost"] -
old_metrics["total_cost"]
        objective = (
            marginal
            + self.cfg.risk_lambda * cvar
            +
self.cfg.route_change_penalty
            - self.cfg.carbon_reward_eta
            * max(0.0, self.cfg.carbon_quota - check["emission"])
        )
        row = {
            "路线编号": st.route_id,
            "车型": st.vehicle.cn_name,
            "插入位置": pos + 1,
            "剩余载重 kg":
st.remaining_weight,
            "剩余体积 m3":
st.remaining_volume,
            "可用载重 kg":
st.spare_weight,
            "可用体积 m3":
st.spare_volume,
            "边际成本": marginal,
            "期望迟到罚金": exp_late,
            "CVaR 风险罚金": cvar,
            "综合目标值": objective,
        }
        evaluations.append(row)
        if best is None or objective <
best["综合目标值"]:
            best = row
            best_penalties = penalties
            # 如果没有可插入现有车辆，则新派一辆
            车
            if best is None:

```

```

                candidates = [v for v in
self.data.vehicles if v.weight_cap >= new_stop.weight
and v.volume_cap >= new_stop.volume]
                if new_stop.green and
in_interval(event_time, self.cfg.green_start,
self.cfg.green_end):
                    candidates = [v for v in candidates
if v.energy == "electric"]
                if not candidates:
                    raise RuntimeError("新增订单无
法由任何车辆服务，请检查车辆容量或限行设置。
")
                # 优先新能源重型车，其次容量最接
                近者
                candidates.sort(key=lambda v:
(v.energy != "electric", v.weight_cap))
                v = candidates[0]
                route = Route(route_id=len(routes) + 1,
vehicle=v, stops=[new_stop], depart=event_time)
                metrics = self.cost.evaluate_sequence(v,
route.stops, event_time, include_fixed=True)

StaticHeuristicSolver._fill_route_metrics(route,
metrics)

                routes.append(route)
                decision = {
                    "调度方式": "新增车辆",
                    "路线编号": route.route_id,
                    "车型": v.cn_name,
                    "插入位置": 1,
                    "说明": "现有车辆无可行插入位
置，因此启用新车辆。",
                }
                return routes,
pd.DataFrame(evaluations), decision, []
                # 执行最佳插入。注意必须冻结已完成任
                务，只重算 event_time 后的剩余路径。
                for r in routes:
                    if r.route_id == int(best["路线编号"]):
                        states_after =
self.get_vehicle_states([r], event_time)[0]
                        pos = int(best["插入位置"]) - 1
                        new_remaining = (

```

```

states_after.remaining_stops[:pos]
                + [new_stop]
                +
states_after.remaining_stops[pos:]
            )
            self._combine_dynamic_metrics(r,
states_after, new_remaining)
            break
        decision = {
            "调度方式": "插入现有路径",
            "路线编号": best["路线编号"],
            "车型": best["车型"],
            "插入位置": best["插入位置"],
            "边际成本": best["边际成本"],
            "CVaR 风险罚金": best["CVaR 风险罚
金"],
            "综合目标值": best["综合目标值"],
        }
        return routes, pd.DataFrame(evaluations),
decision, best_penalties
#
=====
=====
# 7. 输出与可视化
#
=====
=====
def summarize_routes(routes: List[Route]) -> dict:
    return {
        "使用车辆数": len(routes),
        "总成本": sum(r.total_cost for r in routes),
        "固定成本": sum(r.fixed_cost for r in
routes),
        "等待成本": sum(r.wait_cost for r in routes),
        "迟到成本": sum(r.late_cost for r in routes),
        "能源成本": sum(r.energy_cost for r in
routes),
        "碳排放成本": sum(r.carbon_cost for r in
routes),
        "总距离 km": sum(r.distance for r in routes),
        "碳排放 kg": sum(r.emission for r in routes),
        "服务任务数": sum(len(r.stops) for r in
routes),
    }

```

```

def routes_to_df(routes: List[Route]) ->
pd.DataFrame:
    rows = []
    for r in routes:
        rows.append({
            "路线编号": r.route_id,
            "车型": r.vehicle.cn_name,
            "能源类型": "燃油" if r.vehicle.energy
== "fuel" else "新能源",
            "出发时间": minute_to_time(r.depart),
            "返回时间":
minute_to_time(r.back_time),
            "客户路径": r.path_text,
            "累计配送重量 kg": r.weight,
            "累计配送体积 m3": r.volume,
            "车辆载重上限 kg":
r.vehicle.weight_cap,
            "车辆容积上限 m3":
r.vehicle.volume_cap,
            "备注": "第三问动态路径中累计配送
量可能包含触发前已完成任务，不等同于同一时刻
车载量",
            "距离 km": r.distance,
            "固定成本": r.fixed_cost,
            "等待成本": r.wait_cost,
            "迟到成本": r.late_cost,
            "能源成本": r.energy_cost,
            "碳排放成本": r.carbon_cost,
            "总成本": r.total_cost,
            "碳排放 kg": r.emission,
        })
    return pd.DataFrame(rows)
def states_to_df(states: List[VehicleState]) ->
pd.DataFrame:
    rows = []
    for s in states:
        rows.append({
            "路线编号": s.route_id,
            "车型": s.vehicle.cn_name,
            "能源类型": "燃油" if s.vehicle.energy
== "fuel" else "新能源",
            "当前节点": s.current_node,
            "可用时间":
minute_to_time(s.available_time),

```

```

        "已完成任务数":
len(s.completed_stops),
        "剩余任务数": len(s.remaining_stops),
        "剩余载重 kg": s.remaining_weight,
        "剩余体积 m3": s.remaining_volume,
        "可用载重 kg": s.spare_weight,
        "可用体积 m3": s.spare_volume,
    })
    return pd.DataFrame(rows)
def save_excel(out_dir: Path, base_routes: List[Route],
dynamic_routes: List[Route],
                before: dict, after: dict, states:
List[VehicleState],
                eval_df: pd.DataFrame, decision:
dict, cancel_info: dict) -> Path:
    file_path = out_dir / "第三问动态重调度结
果.xlsx"
    comparison = pd.DataFrame({
        "指标": list(before.keys()),
        "动态前": list(before.values()),
        "动态后": [after[k] for k in before.keys()],
        "变化量": [after[k] - before[k] for k in
before.keys()],
    })
    events = pd.DataFrame([
        {"事件": "取消订单", **cancel_info},
        {"事件": "新增订单", **decision},
    ])
    with pd.ExcelWriter(file_path,
engine="openpyxl") as writer:
        comparison.to_excel(writer, sheet_name="动
态前后对比", index=False)
        events.to_excel(writer, sheet_name="事件处
理结果", index=False)
        states_to_df(states).to_excel(writer,
sheet_name="触发时刻车辆状态", index=False)
        eval_df.to_excel(writer, sheet_name="候选
插入评估", index=False)

    routes_to_df(dynamic_routes).to_excel(writer,
sheet_name="重调度后路径", index=False)
    routes_to_df(base_routes).to_excel(writer,
sheet_name="动态前路径", index=False)
    return file_path

```

```

def plot_comparison(out_dir: Path, before: dict, after:
dict) -> None:
    setup_chinese_font()
    labels = ["总成本", "总距离 km", "碳排放 kg", "
使用车辆数"]
    x = np.arange(len(labels))
    b = [before[k] for k in labels]
    a = [after[k] for k in labels]
    width = 0.36
    plt.figure(figsize=(9, 5))
    plt.bar(x - width / 2, b, width, label="动态前")
    plt.bar(x + width / 2, a, width, label="动态后")
    plt.xticks(x, labels, rotation=20)
    plt.ylabel("数值")
    plt.title("第三问动态重调度前后核心指标对比
")
    plt.legend()
    plt.tight_layout()
    plt.savefig(out_dir / "第三问_动态前后指标对比
图.png", dpi=220)
    plt.close()
def plot_candidate_cost(out_dir: Path, eval_df:
pd.DataFrame) -> None:
    setup_chinese_font()
    if eval_df.empty:
        return
    top = eval_df.sort_values("综合目标值
").head(15).copy()
    top["方案"] = top["路线编号"].astype(str) + "-" +
top["插入位置"].astype(str)
    plt.figure(figsize=(10, 5))
    plt.bar(top["方案"], top["综合目标值"])
    plt.xlabel("候选方案: 路线-位置")
    plt.ylabel("综合目标值")
    plt.title("新增订单候选插入方案综合目标值对
比")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig(out_dir / "第三问_候选插入方案对比
图.png", dpi=220)
    plt.close()
def plot_cvar(out_dir: Path, penalties: List[float]) ->
None:
    setup_chinese_font()

```

```

if not penalties:
    return
plt.figure(figsize=(8, 5))
plt.hist(penalties, bins=20)
plt.xlabel("迟到罚金/元")
plt.ylabel("频数")
plt.title("第三问 CVaR 风险模拟: 迟到罚金分布")
plt.tight_layout()
plt.savefig(out_dir / "第三问_CVaR 风险分布图.png", dpi=220)
plt.close()
def plot_routes(out_dir: Path, data: DataLoader, routes: List[Route], title: str) -> None:
    setup_chinese_font()
    plt.figure(figsize=(8, 7))
    xs = [c.x for c in data.customers]
    ys = [c.y for c in data.customers]
    colors = ["red" if c.green else "tab:blue" for c in data.customers]
    plt.scatter(xs, ys, c=colors, s=18, label="客户点")
    if 0 in data.nodes:
        dx, dy = data.nodes[0]
    else:
        dx, dy = (0.0, 0.0)
    plt.scatter([dx], [dy], marker="*", s=180, label="仓库")
    for r in routes[:80]:
        pts = [(dx, dy)] + [(s.x, s.y) for s in r.stops] + [(dx, dy)]
        px = [p[0] for p in pts]
        py = [p[1] for p in pts]
        plt.plot(px, py, linewidth=0.6, alpha=0.45)
    circle = plt.Circle((0, 0), 10, fill=False, linestyle="--", linewidth=1.2, label="绿色配送区")
    plt.gca().add_patch(circle)
    plt.xlabel("X 坐标 km")
    plt.ylabel("Y 坐标 km")
    plt.title(title)
    plt.legend()
    plt.axis("equal")
    plt.tight_layout()
    plt.savefig(out_dir / "第三问_动态重调度路径空间分布图.png", dpi=220)

```

```

plt.close()
def zip_outputs(out_dir: Path) -> Path:
    zip_path = out_dir.with_suffix(".zip")
    if zip_path.exists():
        zip_path.unlink()
    with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as zf:
        for p in out_dir.rglob("*"):
            if p.is_file():
                zf.write(p, p.relative_to(out_dir.parent))
    return zip_path
#
=====
# 8. 主程序
#
=====
def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="第三问: 动态重调度求解代码")
    parser.add_argument("--input", default="A 题清洗过后的数据.xlsx", help="清洗后 Excel 文件路径")
    parser.add_argument("--out", default="第三问动态重调度结果", help="输出文件夹")
    parser.add_argument("--event-time", type=float, default=870.0, help="触发时刻, 默认 14:30=870")
    parser.add_argument("--new-weight", type=float, default=500.0, help="新增订单重量 kg")
    parser.add_argument("--new-volume", type=float, default=1.5, help="新增订单体积 m3")
    parser.add_argument("--new-x", type=float, default=5.0, help="新增订单 X 坐标")
    parser.add_argument("--new-y", type=float, default=5.0, help="新增订单 Y 坐标")
    parser.add_argument("--new-tw-start", type=float, default=870.0, help="新增订单时间窗开始")
    parser.add_argument("--new-tw-end", type=float, default=1020.0, help="新增订单时间窗结束")
    parser.add_argument("--cancel-node", type=int, default=0, help="取消客户编号, 0 表示不取消")
    parser.add_argument("--samples", type=int,

```

```

default=200, help="CVaR 蒙特卡洛模拟次数")
    parser.add_argument("--seed", type=int,
default=42, help="随机种子")
    return parser.parse_args()
def main() -> None:
    args = parse_args()
    random.seed(args.seed)
    np.random.seed(args.seed)
    out_dir = mkdir(args.out)
    cfg = Config()
    print("=" * 70)
    print("第三问：突发事件下的动态重调度求解")
    print("=" * 70)
    print(f'触发时刻:
{minute_to_time(args.event_time)}')
    loader = DataLoader(args.input).load()
    # 新增订单作为新节点加入坐标库，距离用欧
氏距离估算
    new_node_id = max(loader.nodes.keys()) + 1
    new_green = math.hypot(args.new_x, args.new_y)
    <= 10.0
    loader.nodes[new_node_id] = (args.new_x,
args.new_y)
    new_stop = Stop(
        node_id=new_node_id,
        x=args.new_x,
        y=args.new_y,
        weight=args.new_weight,
        volume=args.new_volume,
        tw_start=args.new_tw_start,
        tw_end=args.new_tw_end,
        green=new_green,
        label="动态新增订单",
    )
    cost_model = CostModel(loader, cfg)
    static_solver = StaticHeuristicSolver(loader,
cost_model, cfg)
    base_routes = static_solver.solve()
    before = summarize_routes(base_routes)
    print("\n【动态前基础方案】")
    for k, v in before.items():
        print(f" {k}: {v:.2f}" if isinstance(v, float)
else f" {k}: {v}")
    dispatcher = DynamicDispatcher(loader,

```

```

cost_model, cfg)
    states_before =
dispatcher.get_vehicle_states(base_routes,
args.event_time)
    # 复制一份用于动态修改
    import copy
    dynamic_routes = copy.deepcopy(base_routes)
    dynamic_routes, cancel_info =
dispatcher.apply_cancel_event(
        dynamic_routes, args.cancel_node,
args.event_time
    )
    dynamic_routes, eval_df, decision, penalties =
dispatcher.insert_new_order(
        dynamic_routes, new_stop, args.event_time,
args.samples
    )
    after = summarize_routes(dynamic_routes)
    print("\n【动态事件处理结果】")
    print(f" 新增订单节点: {new_node_id}")
    print(f" 新增订单是否位于绿色区: {'是' if
new_green else '否'}")
    for k, v in decision.items():
        print(f" {k}: {v}")
    print("\n【动态后方案】")
    for k, v in after.items():
        print(f" {k}: {v:.2f}" if isinstance(v, float)
else f" {k}: {v}")
    excel_path = save_excel(
        out_dir, base_routes, dynamic_routes, before,
after,
        states_before, eval_df, decision, cancel_info
    )
    plot_comparison(out_dir, before, after)
    plot_candidate_cost(out_dir, eval_df)
    plot_cvar(out_dir, penalties)
    plot_routes(out_dir, loader, dynamic_routes, "第
三问：动态重调度后配送路径空间分布")
    zip_path = zip_outputs(out_dir)
    print("\n 输出文件: ")
    print(f" Excel 结果: {excel_path}")
    print(f" 打包文件: {zip_path}")
    print("=" * 70)
    if __name__ == "__main__":

```


main()